Qwen-3高效微调实战（上）

- **公开课大纲**

Qwen-3高效微调实战（上）
一、大模型微调技术快速入门
1. 微调基础概念介绍
1.1 **微调基本概念**
1.2 **全量微调与高效微调**
1.3 模型微调的优劣势分析
1.4 **高效微调与LoRA、QLoRA**
2. 高效微调的应用场景
3. 主流微调工具介绍
3.1 unsloth
3.2 LLama-Factory
3.3 ms-SWIFT
3.4 ColossalAI
4. 模型性能评估框架EvalScope
5.微调所需软硬件环境说明
6. 公开课课件领取
【专项提升】如何准备微调数据集
Part 1.原理篇
1. 模型内置特殊字符及提示词模板
2. 带有系统提示和Function calling微调数据集格式
3. 带有思考过程的微调数据集结构
4. Qwen3混合推理模型构造微调数据集基本方法
二、Qwen3模型高效微调环境准备
1. Unsloth安装部署
2. vLLM安装部署流程
2.1 vLLM安装部署
2.2 Qwen3模型权重下载
2.3 借助vllm进行模型调用
3. EvalScope安装部署流程
3.1 EvalScope安装流程
3.2 借助EvalScope进行压力测试
3.3 进行模型性能评估
4. wandb安装与注册
4.1 wandb基本说明
4.2 wandb注册与使用
三、Qwen3混合推理模型微调数据集准备
1. OpenMathReasoning 数据集（AIMO-2）
2. FineTome-100k 数据集（Maxime Labonne）
部分课程成果演示

一、大模型微调技术快速入门

1. 微调基础概念介绍

1.1 微调基本概念

所谓大模型微调，指的在已有的大规模预训练模型基础上，通过对标注数据进行训练，进一步优化模型的表现，以适应特定任务或场景的需求。不同于RAG或者Agent技术，通过搭建工作流来优化模型表现，**微调是通过修改模型参数来优化模型能力，是一种能够让模型"永久"掌握某种能力的方法。**

1.2 全量微调与高效微调

而从方法的大类上来划分，微调又可以划分为全量微调：带入全部数据进行微调，和高效微调：只带入部分数据进行微调。毫无疑问，全量微调是一种算力消耗更大、但对模型的能力改造更为彻底的方法，而高效微调则更类似一种"四两拨千斤"的方法，通过修改模型部分参数，来调整模型整体能力。

1.3 模型微调的优劣势分析

尽管模型微调能够通过修改模型参数的方式，永久的修改模型的能力，但这其实是一把双刃剑，如果处理不当，很可能造成模型原始能力的灾难性遗忘、即会导致模型原始能力丢失，对于复杂模型尤其如此。而为了能够满足微调最初目标，我们必须小心谨慎的设计模型微调数据集和微调训练流程，并经过反复多次训练验证，得到一个最佳模型。

1.4 高效微调与LoRA、QLoRA

尽管全量微调可以对模型的能力进行深度改造，但要带入模型全部参数进行训练，需要消耗大量的算力，且有一定的技术门槛。相比之下，在绝大多数场景中，如果我们只想提升模型某个具体领域的能力，那高效微调会更加合适。尽管在2020年前后，深度学习领域诞生了很多高效微调的方法，但现在适用于大模型的最主流的高效微调方法只有一种——LoRA。

LoRA（Low-Rank Adaptation）微调是一种参数高效的微调方法，旨在通过引入低秩矩阵来减少微调时需要调整的参数数量，从而显著降低显存和计算资源的消耗。具体来说，LoRA 微调并不直接调整原始模型的所有参数，而是通过在某些层中插入低秩的适配器（Adapter）层来进行训练。

LoRA的原理：

- 在标准微调中，我们会修改模型的所有权重，而在 LoRA 中，只有某些低秩矩阵（适配器）被训练和调整。这意味着原始模型的参数保持不变，只是通过少量的新参数来调整模型的输出。
- 低秩矩阵的引入可以在显存和计算能力有限的情况下，依然有效地对大型预训练模型进行微调，从而让 LoRA 成为显存较小的设备上的理想选择。

LoRA的优势：

1. **显存优化：** 只需要调整少量的参数（适配器），显著减少了显存需求，适合显存有限的GPU。
2. **计算效率：** 微调过程中的计算负担也更轻，因为减少了需要调整的参数量。
3. **灵活性：** 可以与现有的预训练模型轻松结合使用，适用于多种任务，如文本生成、分类、问答等。

而**QLoRA（Quantized Low-Rank Adaptation）** 则是 LoRA 的一个扩展版本，它结合了 LoRA 的低秩适配器和量化技术。QLoRA 进一步优化了计算效率和存储需求，特别是在极端显存受限的环境下。与 LoRA 不同的是，**QLoRA 会将插入的低秩适配器层的部分权重进行量化（通常是量化为 INT4 或 INT8）**，在保持性能的同时显著降低模型的存储和计算需求。

- **核心思想：** 在 LoRA 的基础上加入量化技术，减少权重表示的位数，从而降低显存和计算需求。QLoRA 结合了低秩适配器和量化的优点，能够在显存有限的设备上进行更高效的微调。
- **量化：** 通过将模型权重量化为低精度（如 INT4），减少内存占用，并提高推理和训练速度。
- 优势：
    - 在显存非常有限的情况下仍能进行微调。

- 可以处理更大规模的模型。
- 适合用于边缘设备和需要低延迟推理的场景。

LoRA 与 QLoRA 二者对比如下

特性	LoRA	QLoRA
核心技术	低秩适配器（Low-Rank Adapters）	低秩适配器 + 量化技术（Low-Rank Adapters + Quantization）
适用场景	显存受限，但设备性能较好	极限显存受限或需要快速推理的设备
计算效率	提高计算效率，减少调整的参数数量	进一步提升效率，减少内存使用并加快推理速度
量化技术	无量化	将权重量化为低精度（如 INT4 或 INT8）
内存消耗	较低，但不如 QLoRA 低	显著降低内存消耗，适合更小的设备
训练复杂度	较简单，适用于大多数微调场景	需要更多的量化和适配工作，但适合超大模型和设备受限场景

> 微调原理参考：【入门】大语言模型常用微调框架介绍|https://www.bilibili.com/video/BV1Yc411g78a/
>
> 【入门】大语言模型常用微调框架介绍｜LoRA&Prefix-Tuning&Prompt-Tuning&P-Tuning v2...
> 2.9万 150 2023-06-16 17:30:00 未经作者授权，禁止转载
>
>
>

2. 高效微调的应用场景

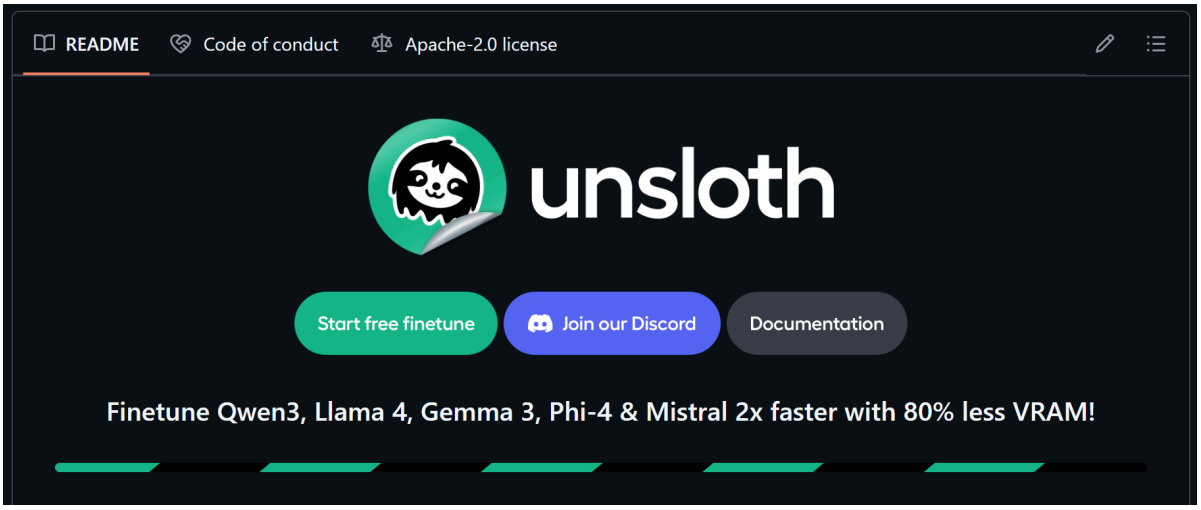
在实际大模型应用场景中，高效微调主要用于以下四个方面：

- **对话风格微调**：高效微调可以用于根据特定需求调整模型的对话风格。例如，针对客服系统、虚拟助理等场景，模型可以通过微调来适应不同的 **语气、礼貌程度** 或 **回答方式**，从而在与用户互动时提供更符合要求的对话体验。通过微调少量的参数（例如对话生成的策略、情感表达等），可以使模型表现出更具针对性和个性化的风格。
- **知识灌注**：知识灌注是指将外部知识或领域特定的信息快速集成到已有的预训练模型中。通过高效微调，模型可以更好地学习新领域的专有知识，而无需重新从头开始训练。例如，对于法律、医疗等专业领域，可以使用少量的标注数据对预训练模型进行微调，帮助模型理解特定行业的术语、规则和知识，进而提升专业领域的问答能力。
- **推理能力提升**：高效微调还可以用于提升大模型的推理能力，尤其是在处理更复杂推理任务时。通过微调，模型能够更加高效地理解长文本、推理隐含信息，或者从数据中提取逻辑关系，进而在多轮推理任务中提供更准确的答案。这种微调方式可以帮助模型在解答复杂问题时，提高推理准确性并减少错误。
- **Agent能力（Function calling能力、或者MCP能力）提升**：在多任务协作或功能调用场景中，高效微调能够显著提升模型的**Agent能力**，使得模型能够有效地与其他系统进行交互、调用外部API或执行特定任务。通过针对性微调，模型可以学会更精准的功能调用策略、参数解析和操作指令，从而在自动化服务、智能助手或机器人控制等领域表现得更加高效和智能。

3. 主流微调工具介绍

在入手学习大模型微调时，首先推荐功能层次封装层次较高的微调四套工具：unsloth、Llama-Factory、ms-SWIFT和ColossalAI。除此之外，也可以借助更加底层的库，如peft、LoRA、transformer等实现高效微调。对于初学者来说，首先使用现成工具来进行微调，四种工具基本说明如下。

3.1 unsloth



- unsloth GitHub主页：https://github.com/unslothai/unsloth

unsloth 是一个专为大型语言模型（LLM）设计的动态量化与微调框架，旨在提高微调效率并减少显存占用。 它通过手动推导计算密集型数学步骤并手写 GPU 内核，实现了无需硬件更改即可显著加快训练速度。

Unsloth supports	Free Notebooks	Performance	Memory use
Qwen3 (14B)	▶ Start for free	2x faster	70% less
GRPO (R1 reasoning)	▶ Start for free	2x faster	80% less
Gemma 3 (4B)	▶ Start for free	1.6x faster	60% less
Llama 3.2 (3B)	▶ Start for free	2x faster	70% less
Phi-4 (14B)	▶ Start for free	2x faster	70% less
Llama 3.2 Vision (11B)	▶ Start for free	2x faster	50% less
Llama 3.1 (8B)	▶ Start for free	2x faster	70% less
Mistral v0.3 (7B)	▶ Start for free	2.2x faster	75% less
Ollama	▶ Start for free	1.9x faster	60% less
DPO Zephyr	▶ Start for free	1.9x faster	50% less

unsloth 与 HuggingFace 生态兼容，可以很容易地transformers、peft、trl 等库结合，以实现模型的监督微调（SFT）和直接偏好优化（DPO），仅需模型的加载方式，无需对现有训练代码进行修改。

- Unsloth动态量化模型：https://unsloth.ai/blog/dynamic-v2

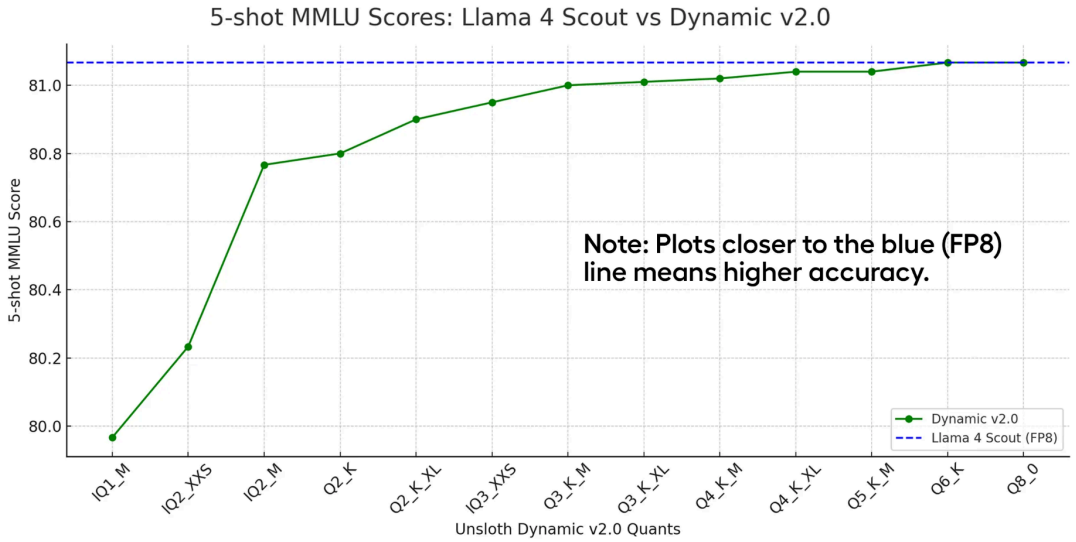
Blog

Unsloth Dynamic 2.0 GGUFs

Apr 24, 2025 • By Daniel & Michael

We're excited to introduce our Dynamic v2.0 quantization method - a major upgrade to our previous quants. This new method outperforms leading quantization methods and sets new benchmarks for 5-shot MMLU and KL Divergence.
This means you can now run + fine-tune quantized LLMs while preserving as much accuracy as possible! You can run the v2.0 GGUFs on any inference engine like llama.cpp, Ollama, Open WebUI etc.
View our Dynamic v2.0 GGUFs here.

Unsloth 的动态量化方法，特别是其最新的 Dynamic 2.0 版本，旨在在尽量减少性能损失的同时显著压缩大型语言模型（LLMs）的体积。对于 Qwen3 模型，尤其是 4-bit 动态量化版本，现有的评测显示其性能下降非常有限，甚至在某些任务上与原始模型相当。具体评测结果如图所示：



而目前，关于 Qwen3 模型在 4-bit 动态量化下的具体性能下降数据尚不全面。然而，根据最近的一项研究，Qwen3 模型在4bit动态量化时，仅损失不到1%的性能。

An Empirical Study of Qwen3 Quantization

Xingyu Zheng, Yuye Li, Haoran Chu, Yue Feng, Xudong Ma, Jie Luo, Jinyang Guo, Haotong Qin, Michele Magno, Xianglong Liu

The Qwen series has emerged as a leading family of open-source Large Language Models (LLMs), demonstrating remarkable capabilities in natural language understanding tasks. With the recent release of Qwen3, which exhibits superior performance across diverse benchmarks, there is growing interest in deploying these models efficiently in resource-constrained environments. Low-bit quantization presents a promising solution, yet its impact on Qwen3's performance remains underexplored. This study conducts a systematic evaluation of Qwen3's robustness under various quantization settings, aiming to uncover both opportunities and challenges in compressing this state-of-the-art model. We rigorously assess 5 existing classic post-training quantization techniques applied to Qwen3, spanning bit-widths from 1 to 8 bits, and evaluate their effectiveness across multiple datasets. Our findings reveal that while Qwen3 maintains competitive performance at moderate bit-widths, it experiences notable degradation in linguistic tasks under ultra-low precision, underscoring the persistent hurdles in LLM compression. These results emphasize the need for further research to mitigate performance loss in extreme quantization scenarios. We anticipate that this empirical analysis will provide actionable insights for advancing quantization methods tailored to Qwen3 and future LLMs, ultimately enhancing their practicality without compromising accuracy. Our project is released on this https URL and this https URL.

> 论文地址：https://arxiv.org/abs/2505.02214

这也使得Unsloth的动态量化模型成为个人配置下的最佳微调工具。

不过需要注意的是，动态量化由利也有弊，其好处在于可以极大程度压缩模型运行所需占用的显存大小，同时几乎不损失性能，但问题在于动态量化的模型，无论是推理还是微调，只能单卡运行，这就使得其吞吐量有限，无法在一台物理机上实现多GPU并行从而扩大吞吐量。

主要功能点：

- **高效微调：** unsloth 通过深度优化，使 LLM 的微调速度提高 2-5 倍，显存使用量减少约 80%，且准确度无明显下降。
- **广泛的模型支持：** 目前支持的模型包括目前各类主流模型，用户可以根据需求适合的模型进行微调。
- **兼容性：** unsloth 与 HuggingFace态系统兼容，用户可以轻松将其与 traformers、peft、l 等库结合，实现模型的监督微调（SFT）和直接偏好优化（DPO），仅需修改模型的加载方式，无需对现有训练代码进行过多修改。

- **内存优化：** 通过 4 位和 16 位的 QLoRA/LoRA 微调，unsloth 显著了显存占用，使得在资源受限的环境中也能大的微调。

unsloth核心优势：

- **显著提升微调效率：** 相比传统方法，Unsloth采用独家4bit动态量化技术，能够在更短的时间内完成微调任务，节省时间成本。
- **降低硬件要求：** 通过优化显存使用，用户可以在显存较小的 GPU 上进行大模型的微调，降低了硬件门槛。
- **开源免费：** Unsloth 提供开源版本，用户可以在 Google Colab 或 Kaggle Notebooks 上免费试用，方便上手体验。

总的来说，unsloth 为大型语言模型的微调提供了高效、低成本的解决方案，适合希望在有限资源下进行模型微调的开发者和研究人员。

3.2 LLama-Factory

- LLama-Factory GitHub主页：https://github.com/hiyouga/LLaMA-Factory

LLaMA-Factory 是一个统一且高效的微调框架，旨在为超过 100 种大型语言模型（LLMs）和视觉语言模型（VLMs）提供便捷的微调支持。 用户能够灵活地定制模型以适应各种下游任务。

主要功能和特点：

- **广型支持：** LLaMA-Factory 支持对 100 多LLMs 和 VLMs 进行微调，包括最新的模型版本，如 Llama 3、GLM-4、Mistral Small、PaliGemma2 等。
- **高效的微调方法：** 框架集成了多nk Adaptation）、QRA（Quantized LoRA）等，以提高训练速度并减少显存占用。
- **多模态任务支持：** 除了传统的文本任务外，LLaMA-Factory 还支频识别、音频理解等多种任务类型。
- **实验监控：** 提供了丰富的实验监控工具，如 LlamaBoard、TensorBoard、Wandb、MLflow、练过程。
- **快速：** 框架提供了类似 OpenAI 风格的 API、Gradio UI 和命令行界面，并结合 vLLM worker，实现了高效的推理能力。

3.3 ms-SWIFT

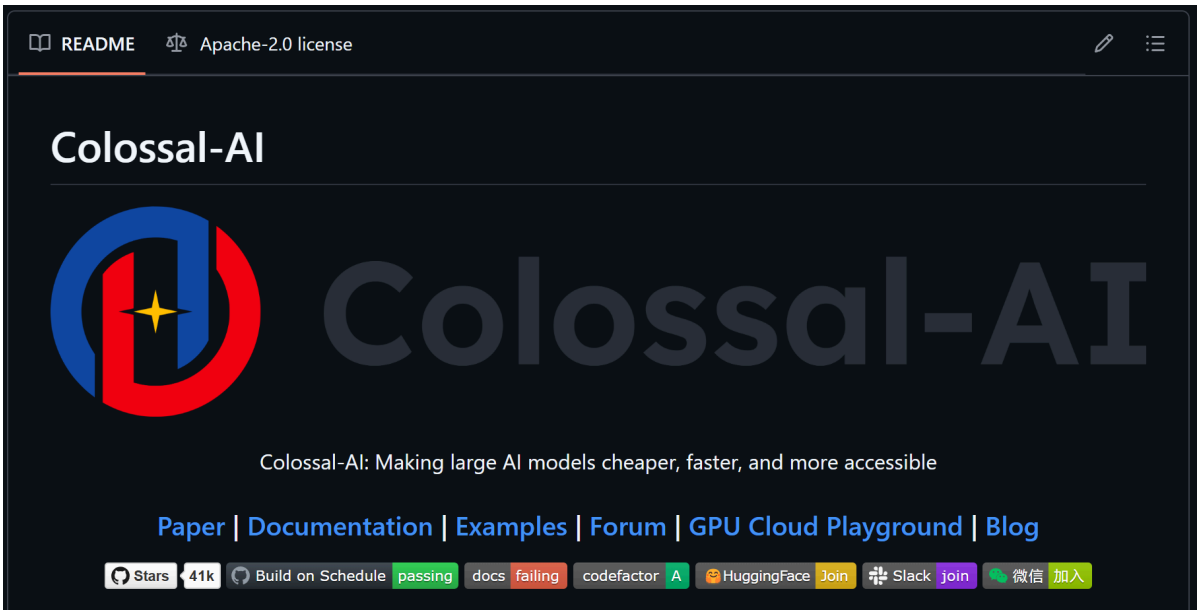
- ms-SWIFT GitHub项目主页：https://github.com/modelscope/swift

ms-swift（Scalable lightWeight Infrastructure for Fine-Tuning）是由魔搭社区（ModelScope）开发的高效微调和部署框架，旨在为研究人员和开发者提供一站式的大模型与多模态大模型的训练、推理、评测、量化和部署解决方案。 的模型支持：** ms-swift 支持超过 450 种大型模型（LLMs）和 150 多种多模态大模型（MLLMs）的训练和部署**，包括最新的模型版本，如 Qwen2.5、InternLM3、GLM4、Llama3.3、Mistral、DeepSeek-R1、Yi1.5、Baichuan2、Gemma2 等，以及多模态模型如 Qwen2.5-VL、Qwen2-Audio、Llama3.2-Vision、Llava、InternVL2.5 等。

- **多样化的训练技术：** 框架集oRA、Llama-Pro、LonoRA、GaLore、Q-GaLore、LoRA+、LISA、DoRA、FourierFt、ReFT、UnSloth 和 Liger 等，满足不同的微调需求。
- **轻量级微调：** 支持多种轻量级微调方法，如 LoRA、QLoRA、DoLLaMAPro、Adapt、GaLore、Q-Galore、LISA、UnSloth、Liger-Kernel 等，降低显存和计算资源的消耗。
- **分布式训练：** 支持分布式数据并行（DDP）、DeepSpeed ZeRO2/ZeRO3、FSDP 等技术，提升推理加速：** 提供 BNBWQ、GPTQ、AQLM、HQQ、EETQ 等量化方法，并支持使用 vLLM 和 LMDeploy 对推理、评测和部署 支持图像、视频和语音等多种模态型训练，涵盖 VQA、Caption、OCR、Grounding 等任务。
- **用户友好的界面：** 提供基于 Gradio 的 We和量化操作，简化了大模型的全链路流程。

3.4 ColossalAI

- ColossalAI GitHub项目主页：https://github.com/hpcaitech/ColossalAI



Colossal-AI 是一个高效的分布式人工智能训练系统，旨在最大化提升人工智能训练效率，同时最小化训练成本。作为深度学习框架的内核，Colossal-AI 提供了自动超高维并行、大规模优化库、自适应任务调度、内存优化以及最新模型复现等前沿技术。与英伟达的 Megatron-LM 相比，Colossal-AI 仅需一半数量的 GPU 即可完成 GPT-3 训练，半小时内预训练 ViT-Base/32，并在两天内训练完 15 亿参数的 GPT 模型。此外，Colossal-AI 提供了多种并行技术，如数据并行、流水线并行和张量并行，以加速模型训练。 □cite□turn0search1□该项目自开源以来，迅速登上 GitHub 热榜，成为解放 AI 生产力的最佳选择。

并且，ColossalAI也是目前唯一支持DeepSeek R1非量化模型高效微调的框架，仅需4个节点、8卡A100服务器即可完成DeepSeek R1高效微调。

February 19, 2025

Shocking Release! DeepSeek 671B Fine-Tuning Guide Revealed—Unlock the Upgraded DeepSeek Suite with One Click, AI Players Ecstatic!

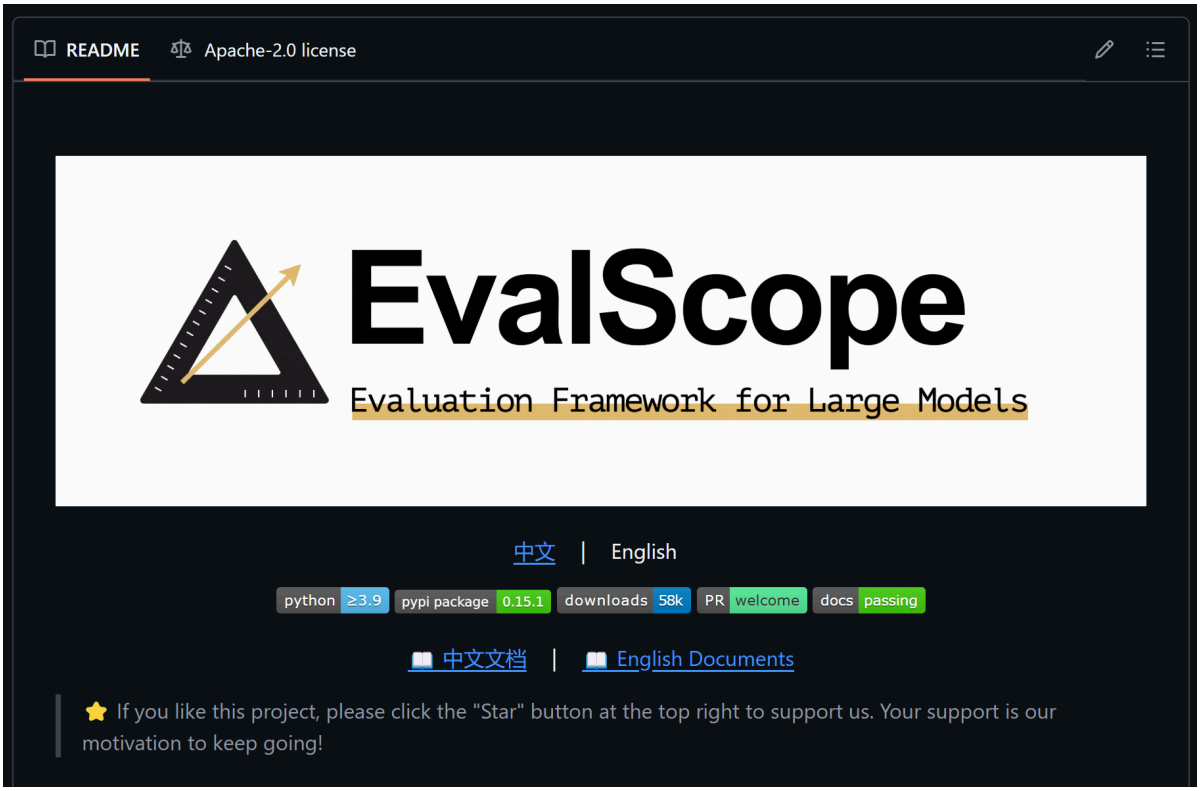
3 minute read

> 注一：若是强化学习训练，则推荐veRL和OpenRLHF等框架。
>
> 注二：其他更底层微调框架推荐

框架	优势	适用场景
Hugging Face	高度兼容，易用，文档丰富	一般 NLP 任务，模型选择丰富
LoRA	显存节省，减少微调计算量	显存有限的设备，微调大规模模型
PEFT	高效微调，低计算开销	资源有限的环境，适合大规模预训练模型的微调
DeepSpeed	大规模分布式训练，显存优化	超大规模训练，多卡分布式训练
AdapterHub	低资源消耗，快速微调	多任务微调，资源有限的环境
Alpaca-LoRA	生成任务优化，LoRA 技术结合	对话生成、文本生成
FastChat	对话系统微调，快速集成	对话生成任务，尤其是对 ChatGPT 等模型微调
FairScale	大规模分布式训练优化，自动化优化	多卡分布式训练，大规模微调

4. 模型性能评估框架EvalScope

- 项目地址：https://github.com/modelscope/evalscope

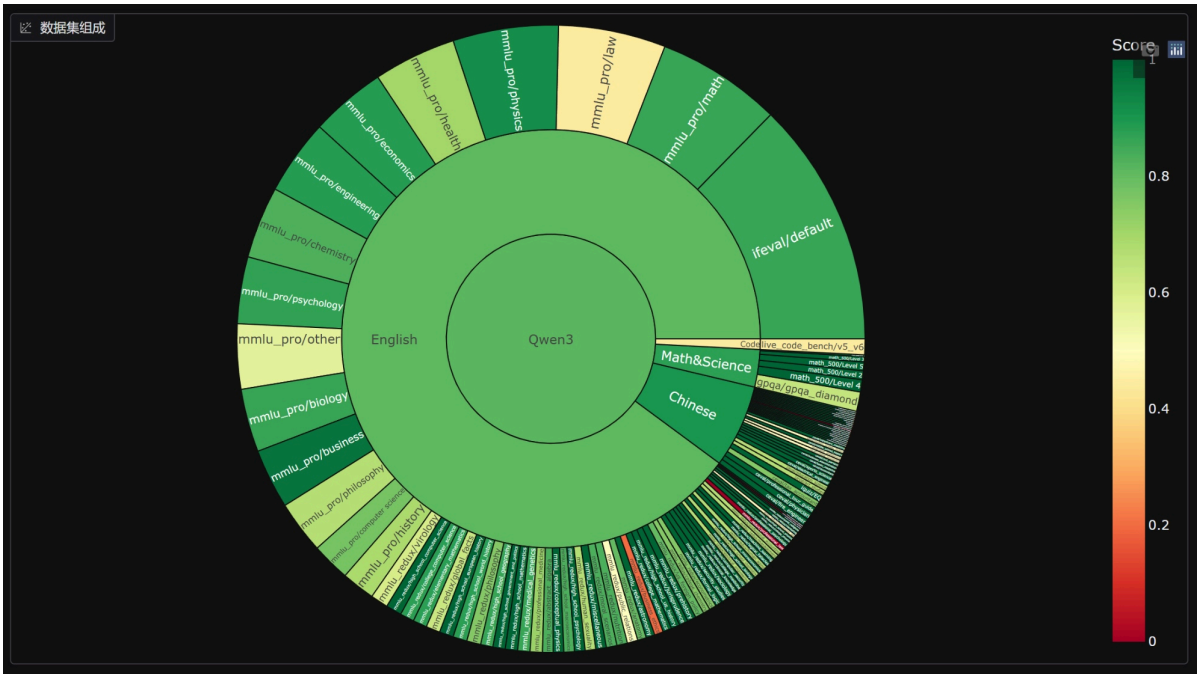


EvalScope 是由阿里巴巴魔搭社区（ModelScope）推出的一款开源模型评估框架，旨在为大语言模型（LLM）和多模态模型提供统一、系统化的性能评估方案。该框架具备高度的自动化和可扩展性，适用于研究机构、工业界以及模型开发者在模型验证与性能对比场景中的广泛需求。

EvalScope 的核心功能和特点包括：

1. **丰富的评测基准覆盖**：框架内置多种权威评测数据集，涵盖中英文通用知识问答（如 MMLU、CMMLU、C-Eval）、数学推理（如 GSM8K、MATH）、常识判断（如 HellaSwag、ARC）、代码生成（如 HumanEval）等多个方向，支持对模型能力进行多维度评估。
2. **多样的评估模式支持**：EvalScope 提供三种灵活的评估模式，包括单模型评估模式（Single）、基于基线的两两对比模式（Pairwise-Baseline）、以及全模型两两对比模式（Pairwise-All），可满足从快速诊断到全面对比的不同使用场景。
3. **统一的模型接入接口**：框架对不同类型的模型提供统一的调用方式，兼容 HuggingFace、本地部署模型及 API 远程调用，支持标准的 `generate` 与 `chat` 接口，大大降低了模型集成的复杂度。
4. **评估流程高度自动化**：EvalScope 实现了评测任务的全自动执行，包括客观题自动打分、复杂问题使用评审模型辅助判定结果等，支持批量评估与日志记录，极大提升了评估效率与结果一致性。
5. **完善的性能与能力可视化工具**：框架支持生成详细的评估报告和图表，展示模型在不同任务维度下的表现，便于开发者进行横向对比和性能分析。
6. **多后端与评测能力扩展**：EvalScope 可集成多个评测后端，如 OpenCompass、VLMEvalKit、RAGEval 等，支持从单模态到多模态、从语言建模到 RAG 端到端评测的全链路能力。
7. **支持部署性能测试**：除评估模型能力外，EvalScope 还提供服务端推理性能测试工具，涵盖吞吐量、响应时延等关键指标，帮助开发者评估模型的部署实用性。

公开课中我们将借助EvalScope进行模型微调前后的性能对比测试。

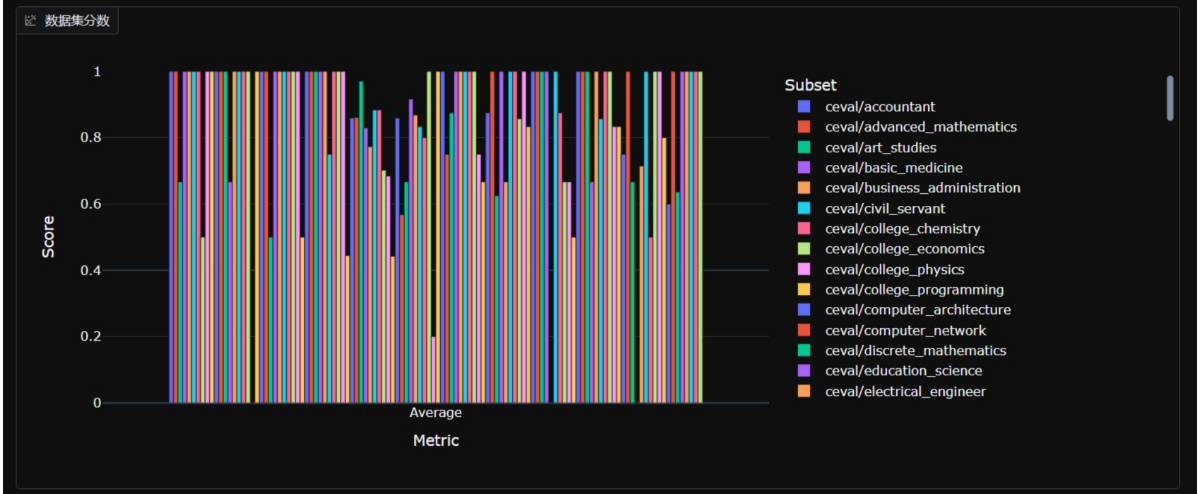


数据集分数

数据集分数表

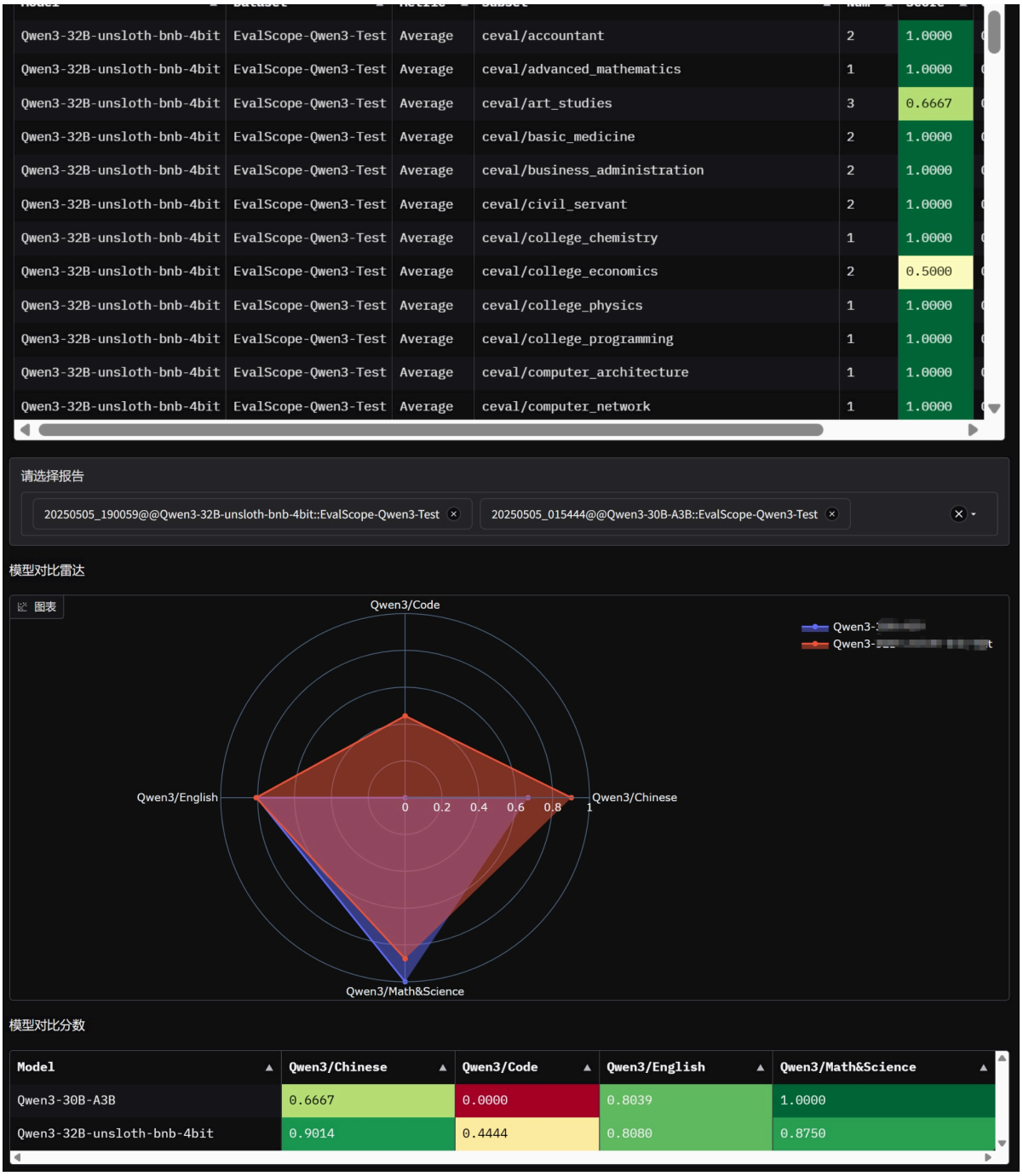
Model	Dataset	Score	Num
Qwen3-32B-unsloth-bnb-4bit	Qwen3/Chinese	0.9014	71
Qwen3-32B-unsloth-bnb-4bit	Qwen3/Code	0.4444	9
Qwen3-32B-unsloth-bnb-4bit	Qwen3/English	0.8080	993
Qwen3-32B-unsloth-bnb-4bit	Qwen3/Math&Science	0.8750	32

数据集分数



数据集分数表

Model	Dataset	Metric	Subset	Num	Score



5.微调所需软硬件环境说明

大模型微调属于大模型进阶类技术，不同于普通的模型对话或搭建基础应用，微调往往需要一定的软硬件条件支持。

- 大模型微调所需硬件一览

  硬件方面，不同尺寸模型、不同精度微调时所需显存如下：

【2025版】大模型GPU硬件配置保姆级指南｜一站式解决深度学习&大模型硬件问题｜大模型推...

2.6万 80 2024-12-16 11:25:00 未经作者授权，禁止转载

> 更多大模型基础硬件配置要求，详见公开课《【2025版】大模型GPU硬件配置保姆级指南》|https://www.bilibili.com/video/BV1VFqZYSEoj

- Qwen3全系列模型微调所需显存一览

  以下是Qwen3系列模型高效微调显存占用预估表：

模型名称	参数量	FP16 微调显存占用	4-bit 动态量化微调显存占用	备注
Qwen3-0.6B	0.6B	~1.2 GB	~0.5 GB	可在低端 GPU 或 CPU 上运行
Qwen3-1.7B	1.7B	~3.4 GB	~1.5 GB	适合入门级部署
Qwen3-4B	4B	~8.0 GB	~3.5 GB	适合中等规模任务
Qwen3-8B	8B	~16.0 GB	~7.0 GB	需要高端消费级 GPU
Qwen3-14B	14B	~28.0 GB	~12.0 GB	可在单张 RTX 4090 上微调
Qwen3-30B-A3B (MoE)	激活参数约 3B	~85.0 GB	暂不支持	激活部分专家参数，资源需求较高
Qwen3-32B	32B	~65.0 GB	~32.0 GB	需要 A100/H100 或多卡并行
Qwen3-235B-A22B (MoE)	激活参数约 22B	~600 GB	暂不支持	超大模型，适合企业级部署，需高端服务器支持

公开课中将采用H800 GPU服务器为大家展示Qwen3 32B 4bit动态量化模型的高效微调流程，同学们可以根据自己实际硬件条件，选取不同模型进行微调，更换不同模型仅需要下载不同模型权重即可，其他流程完全一致。

> 注1：CPU不能进行微调；

> 注2：目前MoE模型只支持4bit普通量化微调，暂不支持动态量化微调。

- 操作系统选择

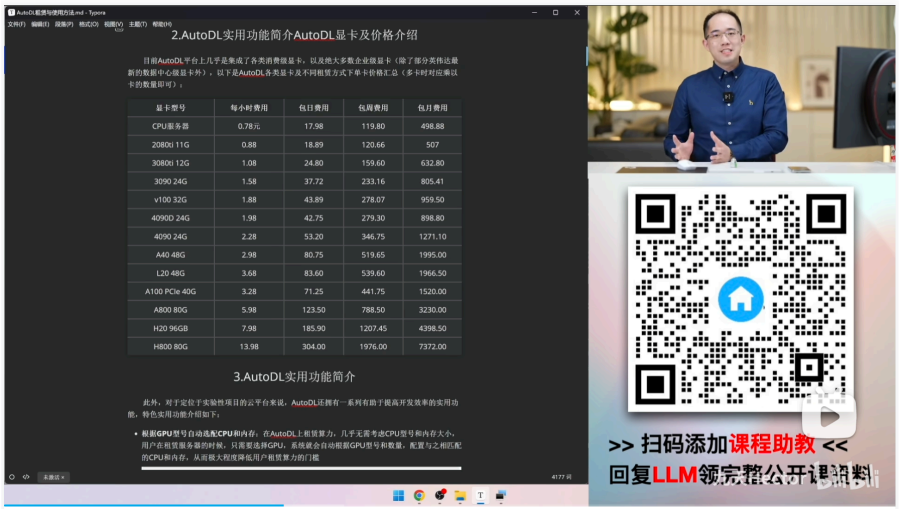
而操作系统方面，由于绝大多数工业场景下微调会涉及多卡微调，目前只有Linux系统对DeepSpeed和其他多卡并行加速库支持较好，因此绝大多数工业场景下都会使用Ubuntu操作系统或CentOS操作系统。本节公开课我们以Ubuntu系统为例来进行高效微调。

若无相关软件环境，本节公开课的相关代码也可以在Windows下运行（本节微调示例不涉及多卡并行）。但若想体验更加真实的工业场景下的微调流程，也可以考虑在AutoDL上租赁显卡并配置Ubuntu服务器来完成操作，例如租赁3090显卡，每小时仅需1.5元，可以微调Qwen3-14B 4bit动态量化模型，运行两小时即可得到结果，仅需不到5元即可完成训练：

> AutoDL相关操作详见公开课：《AutoDL快速入门与GPU租赁使用指南》|https://www.bilibili.com/video/BV1bxB7YYEST/

大模型最佳实验云环境：AutoDL快速入门与GPU租赁使用指南

1785 1 2024-12-17 16:00:00 未经作者授权，禁止转载



6. 公开课课件领取

👇扫码即可领取全部课程资料👇

全部文件 / Qwen3微调实战　　已全部加载，共4个

文件名	大小	类型	修改时间
课件&代码	-	文件夹	2025.05.06 20:13
数据集	-	文件夹	2025.05.06 19:57
模型权重	-	文件夹	2025.05.06 18:39
微调后模型权重	-	文件夹	2025.05.06 14:49

【专项提升】如何准备微调数据集

Part 1.原理篇

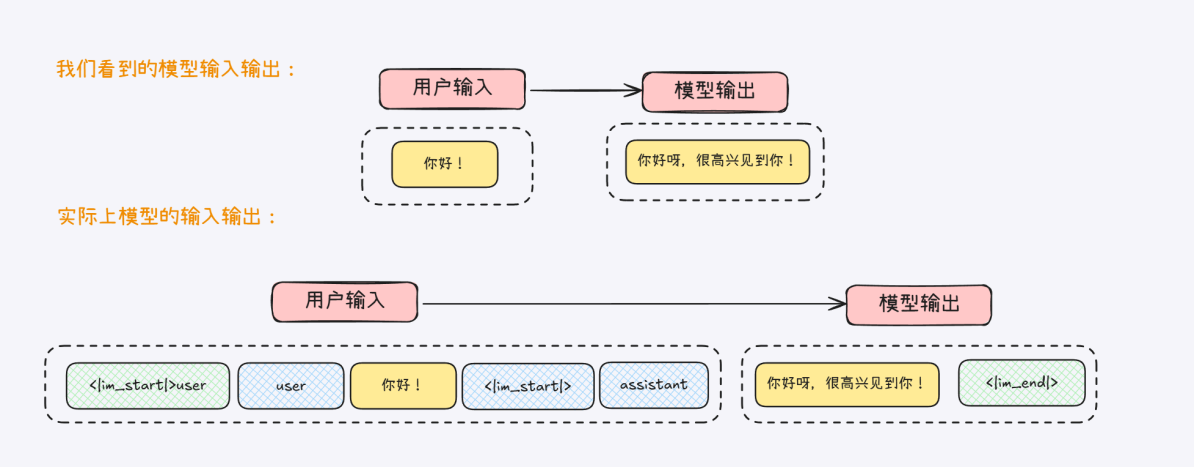
如何创建和选取模型微调数据集，是决定模型微调效果成败的最关键因素，截止目前，已经诞生了各类不同的微调框架和海量的微调数据集，在绝大多数情况下，我们只需要选择不同的微调框架并搭配不同的数据集即可。但伴随着模型能力越来越复杂，包括现阶段很多模型具备了Function calling功能，甚至是具备了推理或者混合推理能力，此时如果希望进行一些复杂功能模型的微调，例如围绕Qwen3模型进行Function calling能力微调、同时还需保留其混合推理能力，此时很多公开数据集或许就无法满足要求了。此外，如果我们希望给模型进行特定领域的知识关注，或者提升模型对于特殊工具组的工具调用准确率，此时就需要手动创建微调数据集了。

而要手动合并或者创建微调数据集，就必须深入了解微调数据集构造背后的原理。本小节内容，就为大家详细介绍创建微调数据集背后的底层原理。

- 本节课件领取

1. 模型内置特殊字符及提示词模板

其实最快速了解构造数据集的方法，是从模型底层原理入手。对于当代大模型来说，普遍需要通过一些特殊字符来标记用户的不同类型输入、系统提示词、以及工具调用或者多模态输入等。而在实际对话过程中，模型对于用户的输入输出是这么进行识别的（以Qwen3为例），一次简答的问答，模型的真实输入和输出如下所示：



其中`<|im_start|>`代表文本开始，而`user`则代表消息身份，用于构建多轮对话，而`<|im_end|>`则代表文本结束，即用户输入结束，而`<|im_start|>`代表新一段文本开始，`assistant`代表接下来由模型创建消息，而`<|im_end|>`同样代表模型创建消息的结束。

而模型其实是通过这样一组特殊字符标记来规范自己的行为，判断当前消息类型，以及通过输出特殊标记来确定停止时间。对于绝大多数模型，我们可以在模型的`tokenizer_config.json`中看到完整的特殊标记符（以及系统提示词模板）：

文件名	大小	类型	修改时间	权限	用户/用户组
._____temp		文件夹	2025/05/04 15:52	drwxr-xr-x	root/root
.msc	917 B	Microsof...	2025/05/04 15:52	-rw-------	root/root
.mv	36 B	MV 文件	2025/05/04 15:52	-rw-r--r--	root/root
config.json	726 B	JSON 源...	2025/05/04 15:45	-rw-r--r--	root/root
configuration.json	73 B	JSON 源...	2025/05/04 15:45	-rw-r--r--	root/root
generation_config.json	239 B	JSON 源...	2025/05/04 15:45	-rw-r--r--	root/root
merges.txt	1.6 MB	文本文档	2025/05/04 15:45	-rw-r--r--	root/root
model.safetensors.ind...	32 KB	JSON 源...	2025/05/04 15:45	-rw-r--r--	root/root
model-00001-of-0000...	3.7 GB	SAFETEN...	2025/05/04 15:52	-rw-r--r--	root/root
model-00002-of-0000...	3.7 GB	SAFETEN...	2025/05/04 15:52	-rw-r--r--	root/root
model-00003-of-0000...	95 MB	SAFETEN...	2025/05/04 15:45	-rw-r--r--	root/root
README.md	16.4 KB	Markdow...	2025/05/04 15:45	-rw-r--r--	root/root
tokenizer_config.json	9.4 KB	JSON 源...	2025/05/04 15:45	-rw-r--r--	root/root
tokenizer.json	10.9 MB	JSON 源...	2025/05/04 15:45	-rw-r--r--	root/root
vocab.json	2.6 MB	JSON 源...	2025/05/04 15:45	-rw-r--r--	root/root

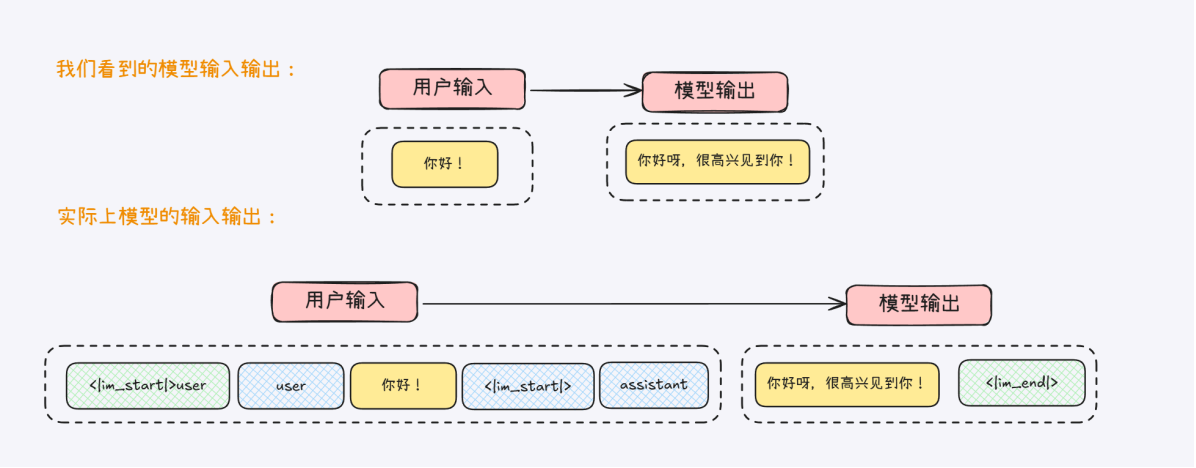
双卡H800_tokenizer_config.json


```
{
  "add_bos_token": false,
  "add_prefix_space": false,
  "added_tokens_decoder": {
    "151643": {
      "content": "<|endoftext|>",
      "lstrip": false,
      "normalized": false,
      "rstrip": false,
      "single_word": false,
      "special": true
    },
    "151644": {
      "content": "<|im_start|>",
      "lstrip": false,
      "normalized": false,
      "rstrip": false,
      "single_word": false,
      "special": true
    },
    "151645": {
      "content": "<|im_end|>",
      "lstrip": false,
      "normalized": false,
      "rstrip": false,
      "single_word": false,
      "special": true
    },
    "151646": {
      "content": "<|object_ref_start|>",
      "lstrip": false,
      "normalized": false,
      "rstrip": false,
      "single_word": false,
      "special": true
    },
    "151647": {
      "content": "<|object_ref_end|>",
      "lstrip": false,
      "normalized": false,
      "rstrip": false,
      "single_word": false,
      "special": true
    },
```


而在实际微调过程中，我们都知道需要有监督的数据集、也就是需要输入QA对来进行微调。以著名的alpaca_zh中文微调数据集来说，其基本格式如下：


```
[
  {
    "instruction": "识别并解释给定列表中的两个科学理论：细胞理论和日心说。",
    "input": "",
    "output": "细胞理论是生物科学的一个理论，它认为所有生命体都是由微小的基本单元—细胞所构成。这是生物学的一个基础理论，认为细胞是所有生物的基本结构和功能单位，所有的生物都是由
  },
  {
    "instruction": "输入三支篮球队的名称并生成一个适当的口号。",
    "input": "输入：俄克拉荷马城雷霆队，芝加哥公牛队，布鲁克林网队。",
    "output": "输出： \"雷霆，公牛和网队：各显神通，角逐群雄!\""
  },
  {
    "instruction": "比较以下两位著名人物，使用特征列表并以句子形式提供输出。",
    "input": "人物1：阿尔伯特·爱因斯坦\n人物2：斯蒂芬·霍金\n特征：智力、教育、对世界的影响",
    "output": "阿尔伯特·爱因斯坦和斯蒂芬·霍金都是拥有极其出色智力的人物。两人都取得过非常高的教育成就，他们推进了科学发展并在世界范围内产生了深远的影响。爱因斯坦以其相对论和质
  },
```


其中的input和output就是输入和输出。例如如下对话：



就可以表示为下列json格式数据集：

```
  {
    "instruction": "",
    "input": "输入：你好。",
    "output": "输出：你好，有什么可以帮到你的？"
  },
```

而在真实的微调过程中，如果是针对Qwen3进行微调，微调脚本会将这条数据集（无论什么格式）转化为如下格式：

```
<|im_start|>user\n你好<|im_end|>\n<|im_start|>assistant\n你好，有什么可以帮到你的？
<|im_end|>
```

而在实际训练过程中，模型就会根据assistant前的内容，学习assistant后面的输出内容。

2. 带有系统提示和Function calling微调数据集格式

在很多场景下，我们还会发现一些带有`instruction`字段的微调数据集，那`instruction`字段是如何带入到微调过程中的呢？

```
[
  {
    "instruction": "识别并解释给定列表中的两个科学理论：细胞理论和日心说。",
    "input": "",
    "output": "细胞理论是生物科学的一个理论，它认为所有生命体都是由微小的基本单元—细胞所构成。这是生物学的一个基础理论，认为细胞是所有生物的基本结构和功能单位，所有的生物都是由
  },
  {
    "instruction": "输入三支篮球队的名称并生成一个适当的口号。",
    "input": "输入：俄克拉荷马城雷霆队，芝加哥公牛队，布鲁克林网队。",
    "output": "输出： \"雷霆，公牛和网队：各显神通，角逐群雄!\""
  },
  {
    "instruction": "比较以下两位著名人物，使用特征列表并以句子形式提供输出。",
    "input": "人物1：阿尔伯特·爱因斯坦\n人物2：斯蒂芬·霍金\n特征：智力、教育、对世界的影响",
    "output": "阿尔伯特·爱因斯坦和斯蒂芬·霍金都是拥有极其出色智力的人物。两人都取得过非常高的教育成就，他们推进了科学发展并在世界范围内产生了深远的影响。爱因斯坦以其相对论和质能
  },
```

答案非常简单，还是依靠特殊字符。例如有一个对话内容如下：

- **系统提示词（instruction）**：你是一名助人为乐的助手。
- **用户输入（input）**：你好，好久不见。
- **助手回复（output）**：是的呀，好久不见，最近有什么有趣的事情要和我分享么？

此时模型的输入和输出如下：

```
<|im_start|>system
你是一名助人为乐的助手。<|im_end|>
<|im_start|>user
你好，好久不见。<|im_end|>
<|im_start|>assistant
是的呀，好久不见，最近有什么有趣的事情要和我分享么？<|im_end|>
```

即会通过`<|im_start|>system...<|im_end|>`来标记系统提示词。实际进行微调时，模型会根据assistant为界，学习assistant之前的文本输入情况下应该如何输出。

更进一步的，如果对话过程中带入了Function calling，此时首先模型会读取提前准备好的tool schema（也可能是自动生成的，例如MCP即可自动创建tool schema）：

```python
tool_schema = [{
    "name": "get_weather",
    "description": "查询指定城市的天气信息",
    "parameters": {
        "type": "object",
        "properties": {
            "location": {
                "type": "string",
                "description": "要查询天气的城市名称"
            }
        },
        "required": ["location"]
    }
}]
```

而假设我们的对话内容如下：

- **系统提示词（instruction）**：你是一名助人为乐的助手。当用户查询天气的时候，请调用get_weather函数进行天气信息查询。
- **用户输入（input）**：你好，请帮我查询下北京天气。
- **助手回复（output）**：{"name": "get_weather", "arguments": {"location": "北京"}}

> 此时回复内容就是一条Function call message

而此时模型真实的输入和输出内容如下：

```
<|im_start|>system
你是一名助人为乐的助手。当用户查询天气的时候，请调用get_weather函数进行天气信息查询。

# Tools

You may call one or more functions to assist with the user query.

You are provided with function signatures within <tools></tools> XML tags:
<tools>
{"name": "get_weather", "description": "查询指定城市的天气信息", "parameters":
{"type": "object", "properties": {"location": {"type": "string", "description":
"要查询天气的城市名称"}}, "required": ["location"]}}
</tools>
```

```
For each function call, return a json object with function name and arguments
within <tool_call></tool_call> XML tags:
<tool_call>
{"name": <function-name>, "arguments": <args-json-object>}
</tool_call>
<|im_end|>
<|im_start|>user
你好，请帮我查询下北京天气。<|im_end|>
<|im_start|>assistant
<tool_call>
{"name": "get_weather", "arguments": {"location": "北京"}}
</tool_call><|im_end|>
```

接下来在进行训练时，模型同样根据assistant前的内容，学习assistant后面的输出内容。不过需要注意的是，由于高效微调调整的参数量较少，因此只能优化模型的Function calling能力，并不能从无到有让模型学会Function calling。

3. 带有思考过程的微调数据集结构

而如果是带有思考链，则一个简单的问答数据如下：

```
{
  "system": "你是一名助人为乐的助手。",
  "conversations": [
    {"role": "user", "content": "你好，好久不见。"},
    {
      "role": "assistant",
      "content": "是的呀，好久不见，最近有什么有趣的事情要和我分享么？",
      "reasoning_content": "好的，用户发来“你好，好久不见！”，我需要回应。首先，用户可能希望得到亲切的回
    }
  ]
}
```

- **系统提示词（instruction）**：你是一名助人为乐的助手。
- **用户输入（input）**：你好，好久不见。
- **助手回复（output）**：好的，用户发来“你好，好久不见！”，我需要回应。首先，用户可能希望得到亲切的回应，所以应该用友好的语气。/n是的呀，好久不见，最近有什么有趣的事情要和我分享么？

此时模型真实的内部输入和输出结果如下：

```
<|im_start|>system
你是一名助人为乐的助手。<|im_end|>
<|im_start|>user
你好，好久不见。<|im_end|>
<|im_start|>assistant
<think>
好的，用户发来“你好，好久不见！”，我需要回应。首先，用户可能希望得到亲切的回应，所以应该用友好的
语气。
</think>

是的呀，好久不见，最近有什么有趣的事情要和我分享么？<|im_end|>
```

模型同样根据assistant前的内容，学习assistant后面的输出内容。也就是说，所谓的思考过程，本质上其实是一种文本响应格式，通过模型训练而来。

最后难度升级，假设是带有思考过程、系统提示词的Function calling流程呢？此时一次对话的基本数据结构如下：

```
"conversations": [
  {
    "role": "user",
    "content": "你好，请帮我查询下北京天气。"
  },
  {
    "role": "assistant",
    "content": "<tool_call>\n{\"name\": \"get_weather\", \"arguments\": {\"location\": \"北京\"}
    "reasoning_content": "好的，用户问北京今天的天气，我应该尝试调用工具 get_weather，并将参数设置为北
  }
```

内容如下：

- **系统提示词（instruction）**：你是一名助人为乐的助手。当用户查询天气的时候，请调用get_weather函数进行天气信息查询。
- **用户输入（input）**：你好，请帮我查询下北京天气。
- **助手回复（output）**：好的，用户问北京今天的天气，我应该尝试调用工具get_weather，并将参数设置为北京。/n{"name": "get_weather", "arguments": {"location": "北京"}}

而此时模型的真实输入和输出如下：

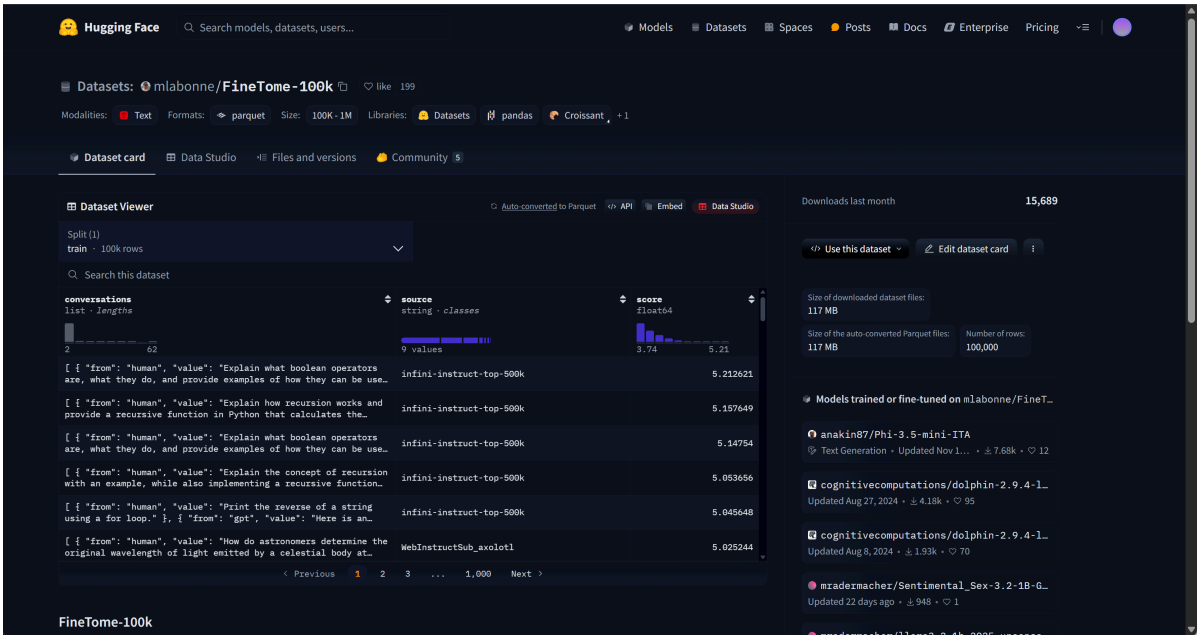
```
<|im_start|>system
你是一名助人为乐的助手。当用户查询天气的时候，请调用get_weather函数进行天气信息查询。

# Tools

You may call one or more functions to assist with the user query.

You are provided with function signatures within <tools></tools> XML tags:
<tools>
{"name": "get_weather", "description": "查询指定城市的天气信息", "parameters":
{"type": "object", "properties": {"location": {"type": "string", "description":
"要查询天气的城市名称"}}, "required": ["location"]}}
</tools>

For each function call, return a json object with function name and arguments
within <tool_call></tool_call> XML tags:
<tool_call>
{"name": <function-name>, "arguments": <args-json-object>}
</tool_call>
<|im_end|>
<|im_start|>user
你好，请帮我查询下北京天气。<|im_end|>
<|im_start|>assistant
<think>
好的，用户问北京今天的天气，我应该尝试调用工具 get_weather，并将参数设置为北京。
</think>

<tool_call>
```

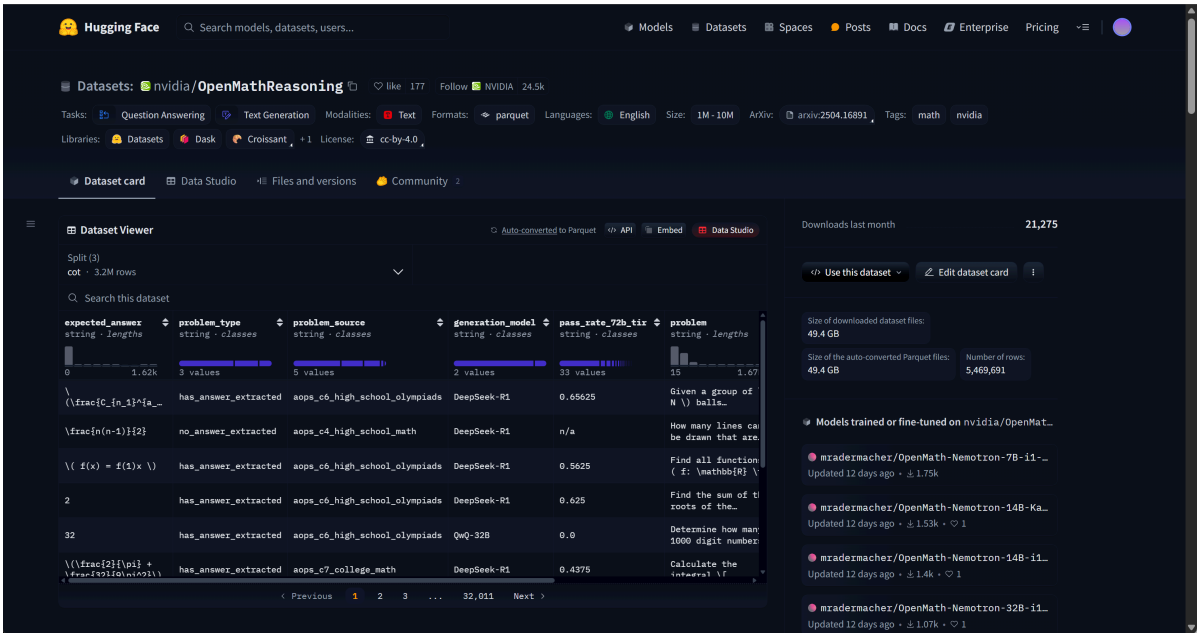
```
{"name": "get_weather", "arguments": {"location": "北京"}}
</tool_call><|im_end|>
```

模型同样根据assistant前的内容，学习assistant后面的输出内容。由此可见，模型拥有不同功能的背后，其实源于不同格式的训练数据集的训练。而对于Qwen3这种模型来说，同时拥有Function calling、混合推理等功能，属于功能非常复杂的模型了。在实际微调过程中，稍有不慎就会令其丧失原有能力。

4. Qwen3混合推理模型构造微调数据集基本方法

在了解了微调数据集结构背后的基本原理后，接下来的问题是应该如何构造微调数据集呢？一般来说我们可以在huggingface、ModelScope或llama-factory中挑选合适的数据集，并根据实际情况进行组装。例如围绕Qwen3模型的高效微调，为了确保其仍然保留混合推理能力，我们可以考虑在微调数据集中加入如普通对话数据集FineTome（https://huggingface.co/datasets/mlabonne/FineTome-100k），以及带有推理字段的数学类数据集OpenMathReasoning（https://huggingface.co/datasets/nvidia/OpenMathReasoning），并围绕这两个数据集进行拼接，从而在确保能提升模型的数学能力的同时，保留非推理的功能。同时还需要在持续微调训练过程中不断调整COT数学数据集和普通文本问答数据集之间的配比，以确保模型能够在提升数学能力的同时，保留混合推理的性能。





此外，更多关于高效微调数据集选取和创建的相关内容，欢迎报名我主讲的《2025大模型Agent智能体开发实战》(5月班)：https://ix9mq.xetslk.com/s/3u765N。

其中将会有更加详细完整的微调全流程实战讲解，**详细信息扫码添加助教，回复“大模型”，即可领取课程大纲&查看课程详情**👇

二、Qwen3模型高效微调环境准备

接下来开始进行模型高效微调前的环境准备工作。需要注意的是，为了更好的复现企业级模型微调全流程，本次公开课将需要使用如下四项工具：

- **【必须】Unsloth**：高效微调框架，必须安装使用；
- **【可选】vLLM**：模型调度框架，用于验证微调后模型效果，也可以使用ollama或者其他调度框架进行模型微调后效果验证；
- **【可选】EvalScope**：模型评测框架，用于对比微调前后模型性能，也可以通过人工观察进行评估；
- **【可选】wandb**：模型训练数据在线记录工具，用于保存模型训练过程中损失之的变化情况，并监控服务器硬件数据；

若想尽快完成微调，可以只安装Unsloth即可，若希望完整执行完微调、过程监督和效果测试各环节，则需要完整安装完各框架工具。

1. Unsloth安装部署

由于要安装多个项目，因此建议创建虚拟环境以避免依赖冲突。这里首先为Unsloth创建虚拟环境：

```
conda create --name unsloth python=3.11
conda init
source ~/.bashrc
conda activate unsloth
```

然后在虚拟环境中安装Jupyter及Jupyter Kernel：

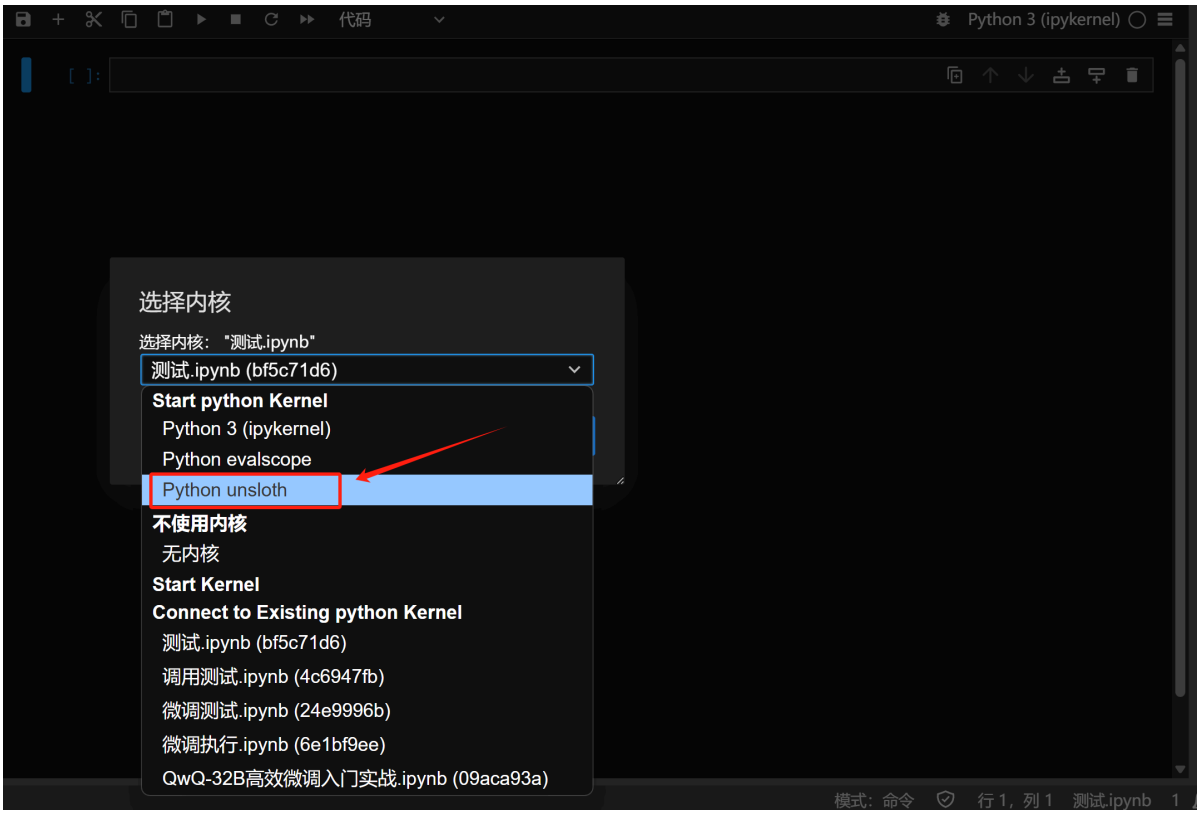
```
conda install jupyterlab
conda install ipykernel
python -m ipykernel install --user --name unsloth --display-name "Python unsloth"
```

然后使用如下命令安装Unsloth：

```
pip install --upgrade --force-reinstall --no-cache-dir unsloth unsloth_zoo
```

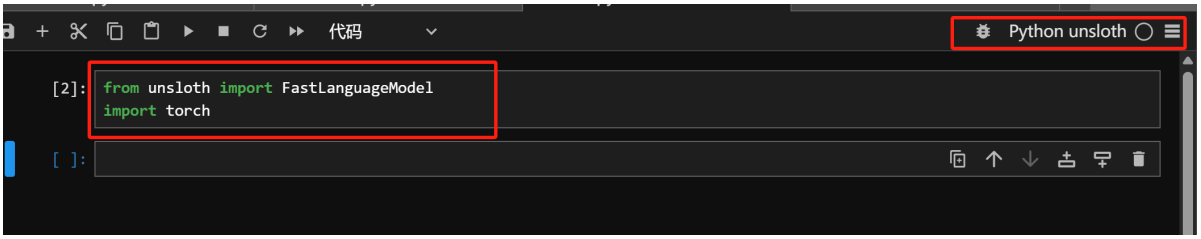
```
root@autodl-container-33284486d8-1f294a77:~# pip install unsloth
Looking in indexes: http://mirrors.aliyun.com/pypi/simple
Collecting unsloth
  Downloading http://mirrors.aliyun.com/pypi/packages/f2/05/6f0835610181f1cc1e85b97faffecabc5791af29595fb07812bbddfa3c04
/unsloth-2025.1.8-py3-none-any.whl (175 kB)
     ---------------------------------------- 175.5/175.5 kB 3.4 MB/s eta 0:00:00
Collecting unsloth_zoo>=2025.1.4 (from unsloth)
  Downloading http://mirrors.aliyun.com/pypi/packages/50/27/9beaf56cfaac7a8e0a32eb310c2b762e786298945113cb523f74a1220969
/unsloth_zoo-2025.1.5-py3-none-any.whl (80 kB)
     ---------------------------------------- 80.2/80.2 kB 5.5 MB/s eta 0:00:00
Requirement already satisfied: torch>=2.4.0 in ./miniconda3/lib/python3.12/site-packages (from unsloth) (2.5.1+cu124)
Collecting xformers>=0.0.27.post2 (from unsloth)
  Downloading http://mirrors.aliyun.com/pypi/packages/37/ac/1aca7e44c93876dbda00e80f79c0bda78bc65e236c68ceb2fc6b26f77df5
/xformers-0.0.29.post2-cp312-cp312-manylinux_2_28_x86_64.whl (44.3 MB)
     ---------------------------------------- 44.3/44.3 MB 16.7 MB/s eta 0:00:00
Collecting bitsandbytes (from unsloth)
  Downloading http://mirrors.aliyun.com/pypi/packages/e4/26/0897c35865a6d7a04b05af5d59558484fda14fa26e5ec922a3bb8d3e0d18
/bitsandbytes-0.45.1-py3-none-manylinux_2_24_x86_64.whl (69.7 MB)
     ---------------------------------------- 35.0/69.7 MB 18.3 MB/s eta 0:00:02
```

安装完成后在任意Jupyter中选择unsloth kernel，即可进入对应的虚拟环境进行代码编写：



可以输入如下代码进行测试

```
from unsloth import FastLanguageModel
import torch
```



2. vLLM安装部署流程

2.1 vLLM安装部署

接下来继续安装vLLM，若是在AutoDL上租赁的服务器，则可以在默认环境中进行安装，其他服务器上建议创建虚拟环境进行安装。虚拟环境创建流程如下（AutoDL不用）：

```
conda create --name vllm python=3.11
conda init
source ~/.bashrc
conda activate vllm
```

vllm主要提供后台的模型调用服务，因此不用安装Jupyter Kernel。

接下来继续进行vllm安装：

```
pip install bitsandbytes>=0.45.3
pip install --upgrade vllm
```

> 注：bitsandbytes是为了适配4bit动态量化模型调用的库

2.2 Qwen3模型权重下载

接下来进行模型权重下载，更多Qwen3模型全尺寸模型本地ollama、vllm、llama.cpp调用及部署方法，详见公开课：https://www.bilibili.com/video/BV1qVG9z4ELJ/

及赋范大模型技术社区文档：

✅ **Qwen-3 @九天Hector**

⭐Qwen-3深度解析！仅需1/3硬件成本，性能超越DeepSeek-R1！（课件）

- Qwen-3深度解析！硬件配置、原生MCP功能介绍、模型选择详解！（视频）

⭐Qwen-3部署与调用详解丨模型详解、transformer原生库调用流程、Qwen3-14B模型接入Ollama（课件）

⭐Qwen-3部署与调用详解丨Qwen3模型接入vLLM、OpenWeb-UI，借助llama.cpp运行Qwen3模型（课件）

- Qwen3硬件配置、模型选择与本地部署完全指南，全面支持MCP，思考模式自由切换！（视频）

⭐Qwen3接入MCP技术实战（上）（课件）

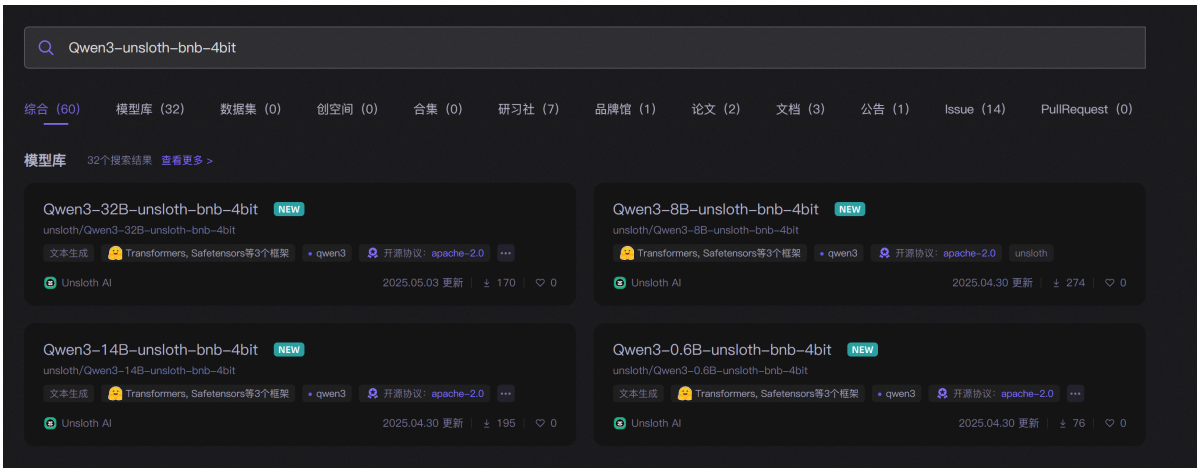
⭐Qwen3接入MCP技术实战（下）（课件）

- Qwen3接入MCP，Qwen3原生MCP能力解析Qwen-Agen接入海量MCP高效搭建智能体！（视频）

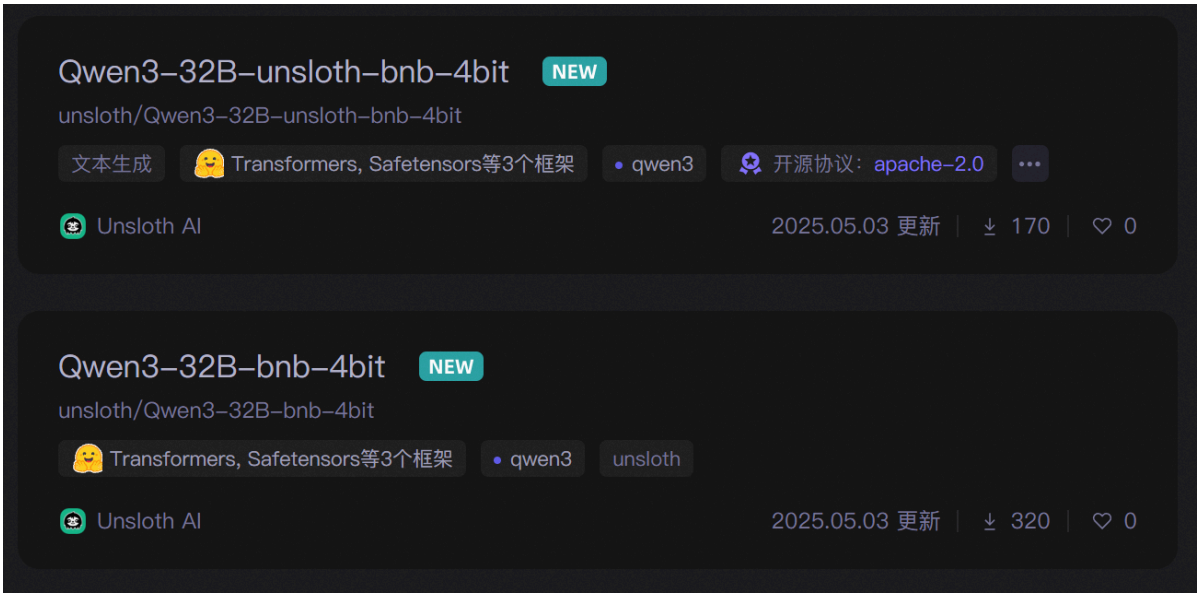
⭐自由控制Qwen3模型的思考模式（课件）

- 彻底控制Qwen3思考过程！Ollama、vLLM、SGLang、LMStudio、Ktransformers开启&关闭思考流程详解！（视频）

而要下载4bit动态量化的Unsloth模型，则可以在魔搭社区上搜索`Qwen3-unsloth-bnb-4bit`：



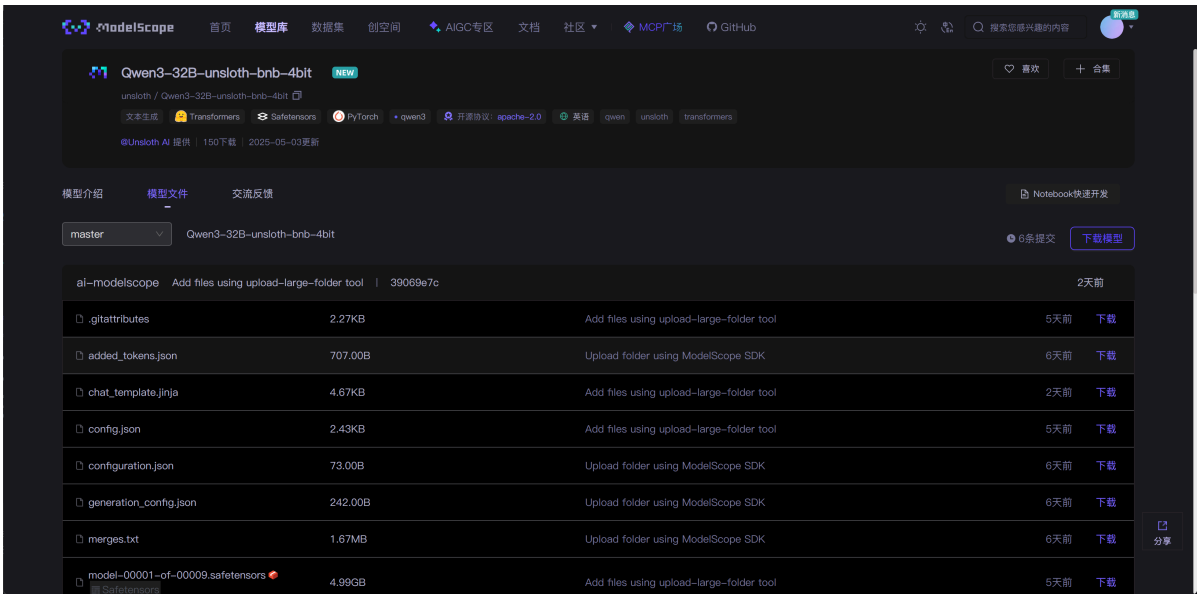
选择模型下载即可。需要注意的是，有时我们会遇到这两种不同的模型，其中带有Unsloth标志的是动态量化模型，而不带unsloth则是普通量化模型，需要注意区分。



> 注，目前最新版vLLM已支持Unsloth动态量化模型，目前Unsloth团队已完成dense模型优化，MoE模型兼容vLLM版目前还未上线。即目前vLLM只支持`unsloth/Qwen3-1.7B-unsloth-bnb-4bit`、`nsloth/Qwen3-4B-unsloth-bnb-4bit`、`unsloth/Qwen3-8B-unsloth-bnb-4bit`、`unsloth/Qwen3-14B-unsloth-bnb-4bit`、`unsloth/Qwen3-32B-unsloth-bnb-4bit`四款模型。

然后安装魔搭社区工具并进行模型下载，以Qwen3-32B-unsloth-bnb-4bit为例（注，**不同模型均可完成微调流程**，大家根据自己硬件配置选择即可）：

https://www.modelscope.cn/models/unsloth/Qwen3-32B-unsloth-bnb-4bit/files

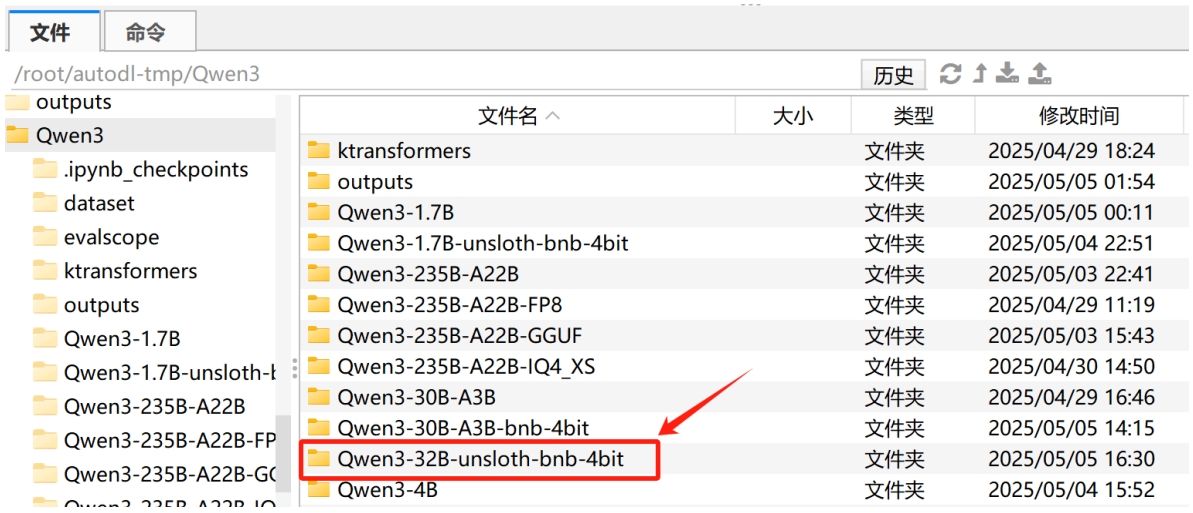


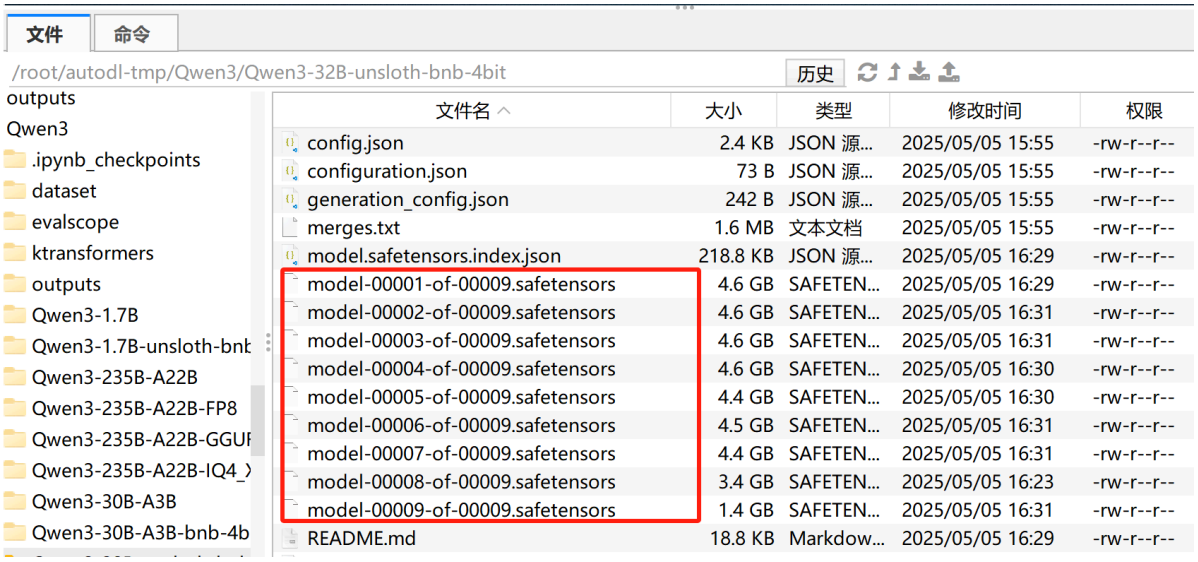
使用如下命令进行下载：

```
pip install modelscope
modelscope download --model unsloth/Qwen3-32B-unsloth-bnb-4bit --local_dir
./Qwen3-32B-unsloth-bnb-4bit
```

```
(base) root@autodl-container-b4de43b42c-6d000661:~/autodl-tmp/Qwen3# modelscope download --model unsloth/Qwen3-32B-unsloth-bnb-4bit
 --local_dir ./Qwen3-32B-unsloth-bnb-4bit
Downloading Model from https://www.modelscope.cn to directory: /root/autodl-tmp/Qwen3/Qwen3-32B-unsloth-bnb-4bit
Downloading [config.json]: 100%|| 2.38k/2.38k [00:00<00:00, 4.79kB/s]
Downloading [generation_config.json]: 100%|| 242/242 [00:00<00:00, 295B/s]
Downloading [configuration.json]: 100%|| 73.0/73.0 [00:00<00:00, 88.4B/s]
Downloading [added_tokens.json]: 100%|| 707/707 [00:00<00:00, 839B/s]
Downloading [chat_template.jinja]: 100%|| 4.56k/4.56k [00:00<00:00, 5.36kB/s]
Downloading [merges.txt]: 100%|| 1.59M/1.59M [00:01<00:00, 1.66MB/s]
Processing 21 items:  29%|| 6.00/21.0 [00:01<00:01, 7.78it/s]
Downloading [model-00007-of-00009.safetensors]:   0%|| 2.00M/4.42G [00:01<52:12, 1.52MB/s]
Downloading [model-00001-of-00009.safetensors]:   0%|| 0.00/4.65G [00:00<?, ?B/s]
Downloading [model-00003-of-00009.safetensors]:   0%|| 3.00M/4.65G [00:01<46:31, 1.79MB/s]
Downloading [model-00004-of-00009.safetensors]:   0%|| 3.00M/4.64G [00:01<30:24, 2.73MB/s]
Downloading [model-00006-of-00009.safetensors]:   0%|| 3.00M/4.48G [00:01<34:17, 2.34MB/s]
Downloading [model-00008-of-00009.safetensors]:   0%|| 2.00M/3.38G [00:01<33:27, 1.81MB/s]
Downloading [model-00002-of-00009.safetensors]:   0%|| 5.00M/4.65G [00:01<22:25, 3.70MB/s]
Downloading [model-00001-of-00009.safetensors]:   0%|| 7.00M/4.65G [00:01<13:27, 6.17MB/s]
```

下载后模型权重如图所示：





2.3 借助vllm进行模型调用

下载完模型权重后，接下来即可测试模型调用流程了，首先开启vllm服务，需要注意的是，Unsloth的动态量化模型只支持单卡调用，最低22G显存即可运行。因此这里是设置的单卡运行vLLM：

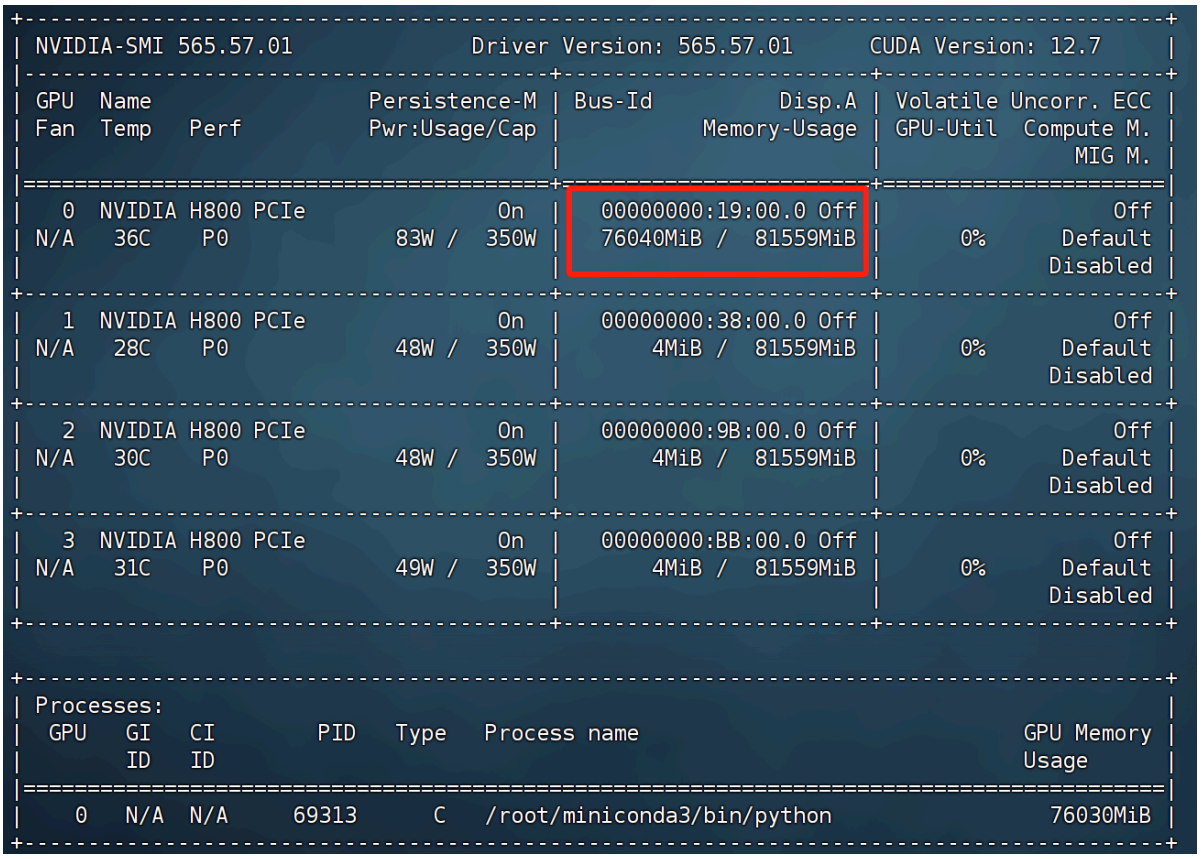
```
# cd /root/autodl-tmp/Qwen3
vllm serve ./Qwen3-32B-unsloth-bnb-4bit --enable-auto-tool-choice --tool-call-parser hermes
```

> 此外，更多关于vLLM调用qwen3参数说明，详见：https://qwen.readthedocs.io/en/latest/deployment/vllm.html#

开启后执行效果如下：

```
INFO 05-05 16:42:56 [launcher.py:36] Route: /docs, Methods: GET, HEAD
INFO 05-05 16:42:56 [launcher.py:36] Route: /docs/oauth2-redirect, Methods: GET, HEAD
INFO 05-05 16:42:56 [launcher.py:36] Route: /redoc, Methods: GET, HEAD
INFO 05-05 16:42:56 [launcher.py:36] Route: /health, Methods: GET
INFO 05-05 16:42:56 [launcher.py:36] Route: /load, Methods: GET
INFO 05-05 16:42:56 [launcher.py:36] Route: /ping, Methods: GET, POST
INFO 05-05 16:42:56 [launcher.py:36] Route: /tokenize, Methods: POST
INFO 05-05 16:42:56 [launcher.py:36] Route: /detokenize, Methods: POST
INFO 05-05 16:42:56 [launcher.py:36] Route: /v1/models, Methods: GET
INFO 05-05 16:42:56 [launcher.py:36] Route: /version, Methods: GET
INFO 05-05 16:42:56 [launcher.py:36] Route: /v1/chat/completions, Methods: POST
INFO 05-05 16:42:56 [launcher.py:36] Route: /v1/completions, Methods: POST
INFO 05-05 16:42:56 [launcher.py:36] Route: /v1/embeddings, Methods: POST
INFO 05-05 16:42:56 [launcher.py:36] Route: /pooling, Methods: POST
INFO 05-05 16:42:56 [launcher.py:36] Route: /score, Methods: POST
INFO 05-05 16:42:56 [launcher.py:36] Route: /v1/score, Methods: POST
INFO 05-05 16:42:56 [launcher.py:36] Route: /v1/audio/transcriptions, Methods: POST
INFO 05-05 16:42:56 [launcher.py:36] Route: /rerank, Methods: POST
INFO 05-05 16:42:56 [launcher.py:36] Route: /v1/rerank, Methods: POST
INFO 05-05 16:42:56 [launcher.py:36] Route: /v2/rerank, Methods: POST
INFO 05-05 16:42:56 [launcher.py:36] Route: /invocations, Methods: POST
INFO 05-05 16:42:56 [launcher.py:36] Route: /metrics, Methods: GET
INFO:     Started server process [68347]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

实际显存占用如下：



> 注，vllm会占用显卡的80%以上显存。实际调用32B 4bit动态量化模型时，最低22G显存即可调用，最低37G显存即可微调。

接下来则可在代码环境中进行调用测试：

```python
from openai import OpenAI

openai_api_key = "EMPTY"
openai_api_base = "http://localhost:8000/v1"

client = OpenAI(
    api_key=openai_api_key,
    base_url=openai_api_base,
)

messages = [
    {"role": "user", "content": "你好，好久不见！"}
]

response = client.chat.completions.create(
    model="./Qwen3-32B-unsloth-bnb-4bit",
    messages=messages,
)

print(response.choices[0].message.content)
```

- 测试Qwen3-32B-unsloth-bnb-4bit模型调用测试

[4]:
```
from openai import OpenAI
```

[5]:
```
openai_api_key = "EMPTY"
openai_api_base = "http://localhost:8000/v1"

client = OpenAI(
    api_key=openai_api_key,
    base_url=openai_api_base,
)
```

[6]:
```
messages = [
    {"role": "user", "content": "你好，好久不见！"}
]
```

[7]:
```
response = client.chat.completions.create(
    model="./Qwen3-32B-unsloth-bnb-4bit",
    messages=messages,
)
```

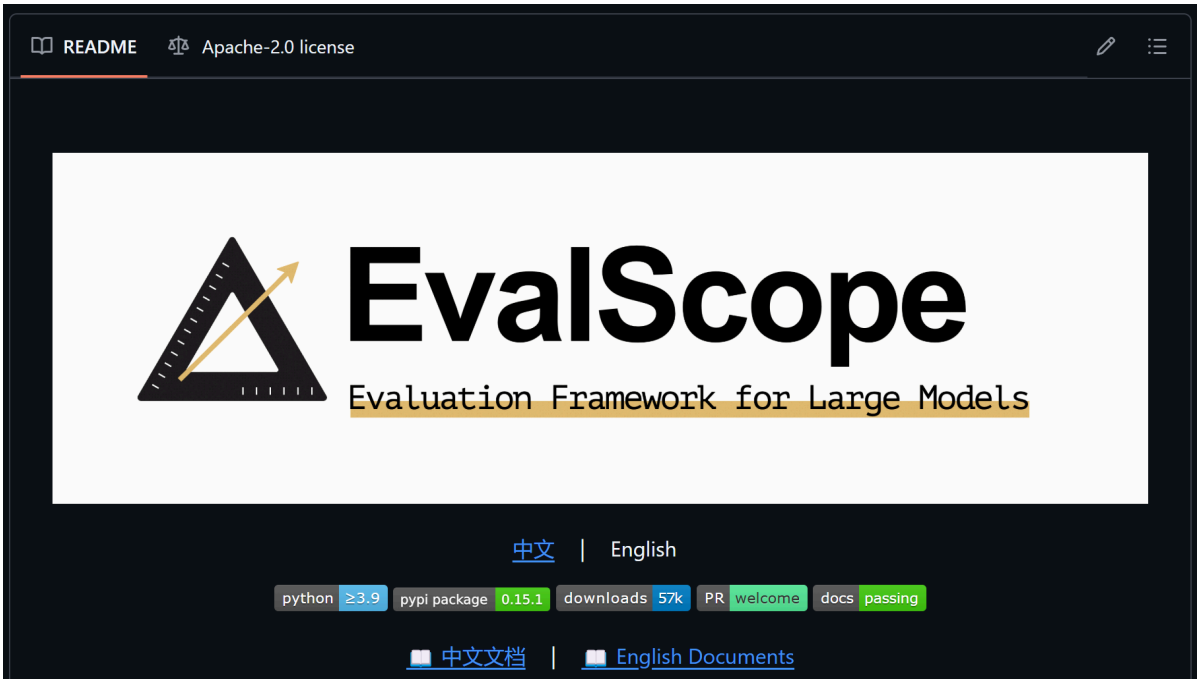
[8]:
```
print(response.choices[0].message.content)
```

```
<think>
好的，用户发来“你好，好久不见！”，我需要友好回应。首先，要表达见到对方的开心，比如“很高兴见到你！”或者“好久不见，真开心！”。然后，可以主动询问对方最近的情况，比如“最近过得怎么样？有什么有趣的事分享吗？”。保持语气亲切自然，避免太正式。同时，注意用户可能想继续聊天，所以可以提出一些话题或活动建议，比如“要不要一起玩个小游戏？”或者“最近有什么新发现吗？”。需要确保回应简洁，不冗长。另外，用户可能希望得到积极的回应，所以可以加入一些表情符号，比如😊，让对话更生动。最后，检查有没有拼写错误，确保信息准确。总之，回应要热情、友好，并引导对话继续。
</think>

好久不见啦！😊 开心得像见到老朋友一样！最近过得怎么样呀？有什么有趣的事情要跟我分享吗？或者想一起玩个小游戏？✨
```

3. EvalScope安装部署流程

3.1 EvalScope安装流程

接下来进行EvalScope安装部署，项目官网：https://github.com/modelscope/evalscope



为了避免和Unsloth的依赖产生冲突，EvalScope需要再单独创建虚拟环境：

```
conda create --name evalscope python=3.11
conda init
source ~/.bashrc
conda activate evalscope

conda install jupyterlab
conda install ipykernel
python -m ipykernel install --user --name evalscope --display-name "Python
evalscope"
```

然后安装对应的库

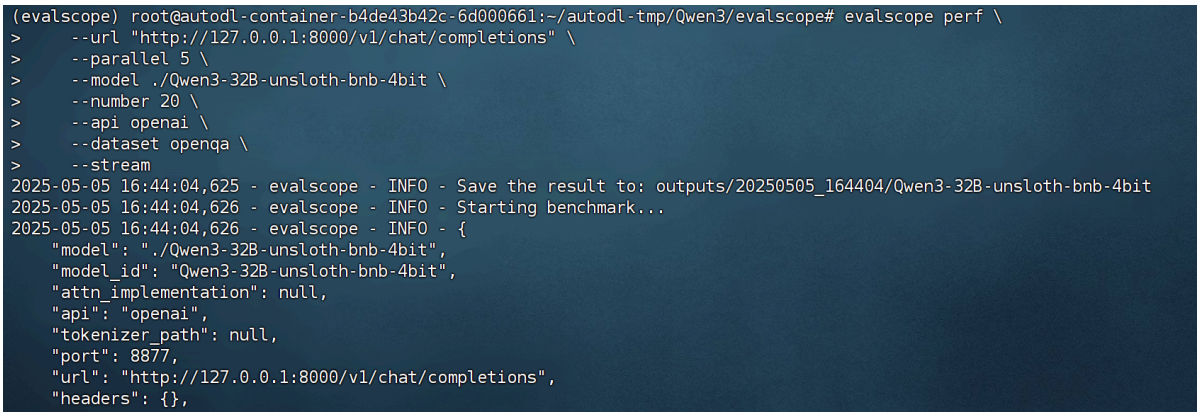
```
pip install evalscope                # 安装 Native backend (默认)
# 额外选项
pip install 'evalscope[opencompass]'   # 安装 OpenCompass backend
pip install 'evalscope[vlmeval]'       # 安装 VLMEvalKit backend
pip install 'evalscope[rag]'           # 安装 RAGEval backend
pip install 'evalscope[perf]'          # 安装 模型压测模块 依赖
pip install 'evalscope[app]'           # 安装 可视化 相关依赖

# 或可以直接输入all，安装全部模块
# pip install 'evalscope[all]'           # 安装所有 backends (Native, OpenCompass,
VLMEvalKit, RAGEval)
```

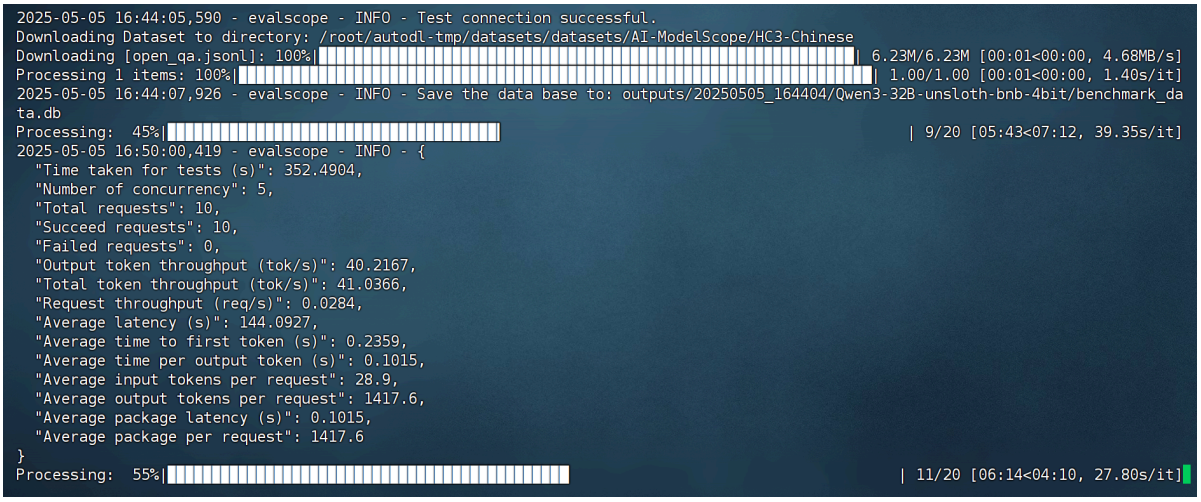
> 源码安装：
>
> ```
> git clone https://github.com/modelscope/evalscope.git
> cd evalscope/
>
> pip install omegaconf
>
> pip install -e .                   # 安装 Native backend
> # 额外选项
> pip install -e '.[opencompass]'   # 安装 OpenCompass backend
> pip install -e '.[vlmeval]'       # 安装 VLMEvalKit backend
> pip install -e '.[rag]'           # 安装 RAGEval backend
> pip install -e '.[perf]'          # 安装 模型压测模块 依赖
> pip install -e '.[app]'           # 安装 可视化 相关依赖
> pip install -e '.[all]'           # 安装所有 backends (Native, OpenCompass,
> VLMEvalKit, RAGEval)
> ```

3.2 借助EvalScope进行压力测试

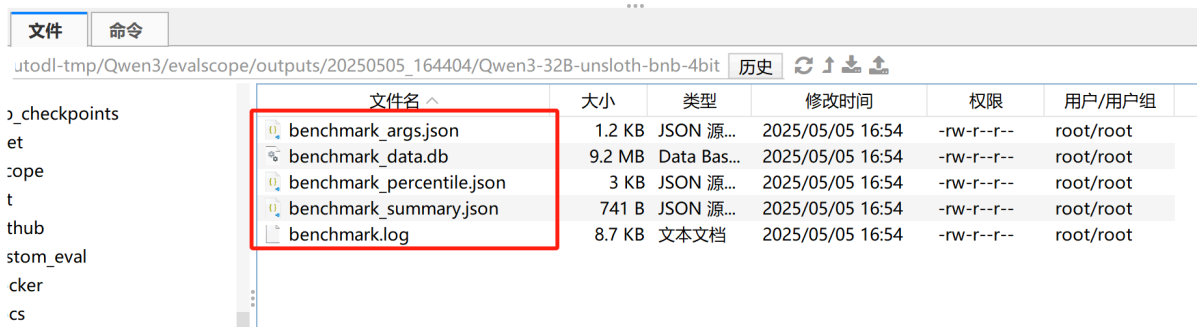
紧接着，我们就可以尝试进行模型的压力测试，可以测试当前4bit动态量化模型在单卡H800上，由vllm调度框架驱动时的实际性能表现：

```
evalscope perf \
    --url "http://127.0.0.1:8000/v1/chat/completions" \
    --parallel 5 \
    --model ./Qwen3-32B-unsloth-bnb-4bit \
    --number 20 \
    --api openai \
    --dataset openqa \
    --stream
```





输出结果保存在：`outputs/20250505_164404/Qwen3-32B-unsloth-bnb-4bit`：



然后即可在summary.json中查看压测的benchmark

```
{
    "Time taken for tests (s)": 632.0044,
    "Number of concurrency": 5,
    "Total requests": 20,
    "Succeed requests": 20,
    "Failed requests": 0,
    "Output token throughput (tok/s)": 45.5503,
```

```
    "Total token throughput (tok/s)": 46.4759,
    "Request throughput (req/s)": 0.0316,
    "Average latency (s)": 142.0679,
    "Average time to first token (s)": 0.2191,
    "Average time per output token (s)": 0.0994,
    "Average input tokens per request": 29.25,
    "Average output tokens per request": 1439.4,
    "Average package latency (s)": 0.0985,
    "Average package per request": 1439.4,
    "Expected number of requests": 20,
    "Result DB path": "outputs/20250505_164404/Qwen3-32B-unsloth-bnb-4bit/benchmark_data.db"
}
```


Benchmarking summary:

Key	Value
Time taken for tests (s)	632.0044
Number of concurrency	5
Total requests	20
Succeed requests	20
Failed requests	0
Output token throughput (tok/s)	45.5503
Total token throughput (tok/s)	46.4759
Request throughput (req/s)	0.0316
Average latency (s)	142.0679
Average time to first token (s)	0.2191
Average time per output token (s)	0.0994
Average input tokens per request	29.25
Average output tokens per request	1439.4
Average package latency (s)	0.0985
Average package per request	1439.4
Expected number of requests	20
Result DB path	outputs/20250505_164404/Qwen3-32B-unsloth-bnb-4bit/benchmark_data.db

Percentile results:

Percentile	TTFT (s)	ITL (s)	TPOT (s)	Latency (s)	Input tokens	Output tokens	Output throughput(tok/s)	Total throughput(tok/s)
10%	0.1998	0.101	0.1013	102.6404	21	1007	9.8255	9.9769
25%	0.2016	0.1012	0.1013	121.5114	26	1197	9.8412	10.0059
50%	0.2028	0.1014	0.1013	148.9654	28	1475	9.8546	10.0649
66%	0.2036	0.1015	0.1014	156.6713	31	1561	9.8559	10.1057
75%	0.2047	0.1016	0.1016	165.1811	34	1748	9.8571	10.1505
80%	0.29	0.1016	0.1016	177.8657	37	1849	9.8578	10.1567
90%	0.291	0.1019	0.1016	207.6119	41	2048	9.8646	10.5866
95%	0.2915	0.1022	0.1016	208.2209	45	2048	16.282	16.4887
98%	0.2915	0.103	0.1016	208.2209	45	2048	16.282	16.4887
99%	0.2915	0.1045	0.1016	208.2209	45	2048	16.282	16.4887

压测结果解释：

📊 压测核心指标解读

指标项	含义	解读
Time taken for tests (s)	总测试耗时	632 秒，表示本次压测运行了约 10 分钟。
Number of concurrency	并发数	使用了 5 个并发请求同时测试，表明模型支持一定程度的并发推理能力。
Total requests	总请求数	总共向模型发起了 20 次推理请求。
Succeed requests / Failed requests	成功与失败数	全部成功（20/20），说明模型稳定性良好。

🚀 吞吐性能

指标项	含义	解读
Output token throughput (tok/s)	每秒生成 token 数	45.55 tok/s，生成速度在 4bit 动态量化大模型中属于正常偏低水平。
Total token throughput (tok/s)	总吞吐（包括输入+输出）	46.48 tok/s，说明输入 token 占比较小，推理主要时间在输出部分。
Request throughput (req/s)	每秒处理请求数	0.0316 req/s，即大约 31 秒才完成一次请求（见下方 latency）。适用于高吞吐非交互式任务。

⏱️ 延迟分析

指标项	含义	解读
Average latency (s)	单次请求平均耗时	142 秒，即每个请求耗时超过 2 分钟，说明单次生成量很大。
Average time to first token (s)	首 token 延迟	0.2191 秒，响应启动速度非常快，体现了量化模型的推理启动效率。
Average time per output token (s)	每个生成 token 的平均耗时	0.0994 秒，略高，意味着模型速度偏慢，可能与显存带宽或激活参数有关。

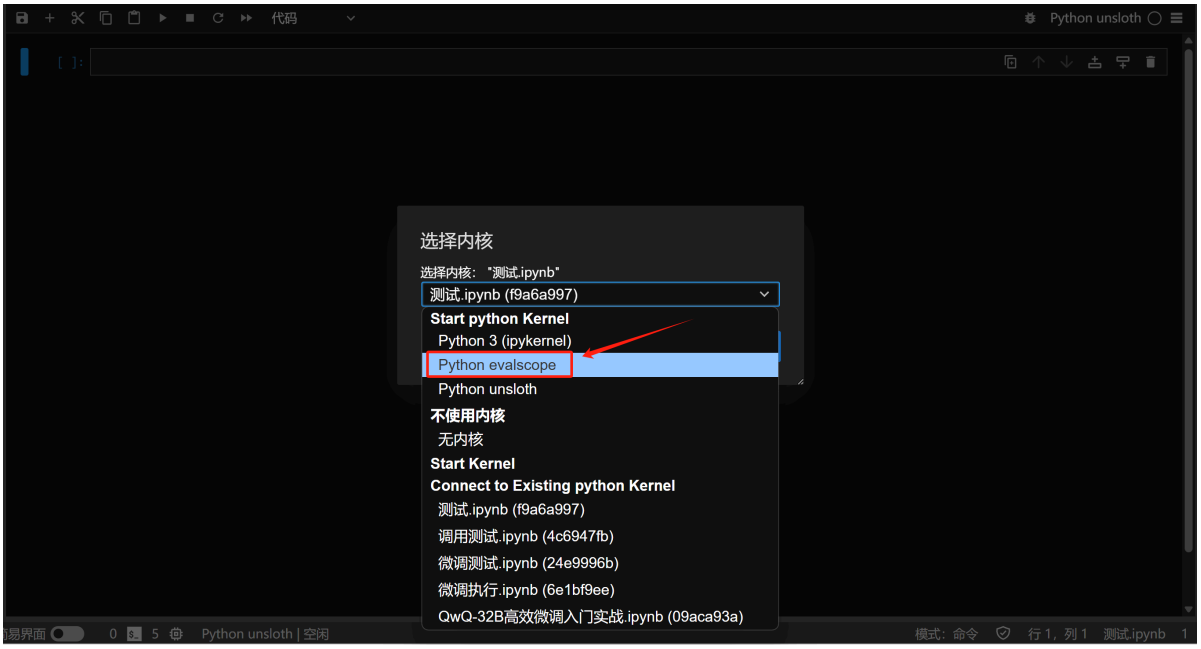
📦 Token 分布与批处理

指标项	含义	解读
Average input tokens per request	每次输入平均 token 数	29.25，说明输入 prompt 很短。
Average output tokens per request	每次输出平均 token 数	1439.4，生成文本非常长，这解释了高 latency 与低 req/s。
Average package latency (s)	批处理延迟	0.0985 秒，表示模型每次接受请求到开始处理之间延迟极短。
Average package per request	每次请求中 token 包含数	同为 1439.4，表明未使用分片或 chunk 输出，模型按完整长度生成。

- **优点：**
  - 模型非常稳定（20 次请求 100% 成功）
  - 首 token 响应迅速（0.21 秒），表明部署结构良好
  - 适合**长文本生成类任务**
- **缺点：**
  - 整体吞吐较低（45 tok/s），属于 32B 大模型的正常范围
  - 单次推理耗时较长（每次平均 142 秒），适用于非实时场景。

3.3 进行模型性能评估

接下来即可进一步进行模型性能评估。这里我们先尝试对其进行初始状态下的性能评估，然后等微调结束后，再进行新一轮的评估，进而对比微调前后模型性能变化情况。这里需要在Jupyter中选择evalscope kernel：



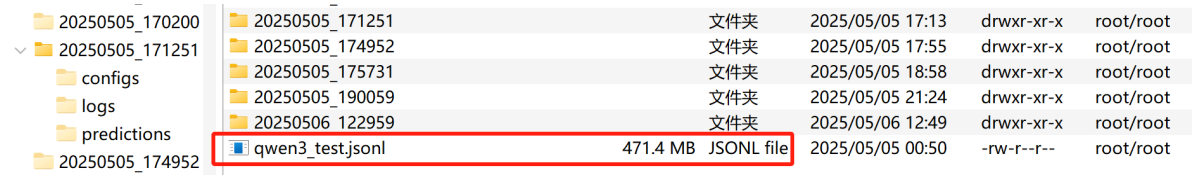
然后运行如下代码：

【可选】数据集构造代码：

```
from evalscope.collections import CollectionSchema, DatasetInfo, WeightedSampler
from evalscope.utils.io_utils import dump_jsonl_data

schema = CollectionSchema(name='Qwen3', datasets=[
    CollectionSchema(name='English', datasets=[
        DatasetInfo(name='mmlu_pro', weight=1, task_type='exam', tags=['en'],
args={'few_shot_num': 0}),
        DatasetInfo(name='mmlu_redux', weight=1, task_type='exam', tags=['en'],
args={'few_shot_num': 0}),
        DatasetInfo(name='ifeval', weight=1, task_type='instruction', tags=
['en'], args={'few_shot_num': 0}),
    ]),
    CollectionSchema(name='Chinese', datasets=[
        DatasetInfo(name='ceval', weight=1, task_type='exam', tags=['zh'], args=
{'few_shot_num': 0}),
        DatasetInfo(name='iquiz', weight=1, task_type='exam', tags=['zh'], args=
{'few_shot_num': 0}),
    ]),
```

```
    CollectionSchema(name='Code', datasets=[
        DatasetInfo(name='live_code_bench', weight=1, task_type='code', tags=
['en'], args={'few_shot_num': 0, 'subset_list': ['v5_v6'], 'extra_params':
{'start_date': '2025-01-01', 'end_date': '2025-04-30'}}),
    ]),
    CollectionSchema(name='Math&Science', datasets=[
        DatasetInfo(name='math_500', weight=1, task_type='math', tags=['en'],
args={'few_shot_num': 0}),
        DatasetInfo(name='aime24', weight=1, task_type='math', tags=['en'], args=
{'few_shot_num': 0}),
        DatasetInfo(name='aime25', weight=1, task_type='math', tags=['en'], args=
{'few_shot_num': 0}),
        DatasetInfo(name='gpqa', weight=1, task_type='knowledge', tags=['en'],
args={'subset_list': ['gpqa_diamond'], 'few_shot_num': 0})
    ])
])

# get the mixed data
mixed_data = WeightedSampler(schema).sample(100000000)  # set a large number to
ensure all datasets are sampled
# dump the mixed data to a jsonl file
dump_jsonl_data(mixed_data, 'outputs/qwen3_test.jsonl')
```

创建好的完整数据集已上传至课件网盘：

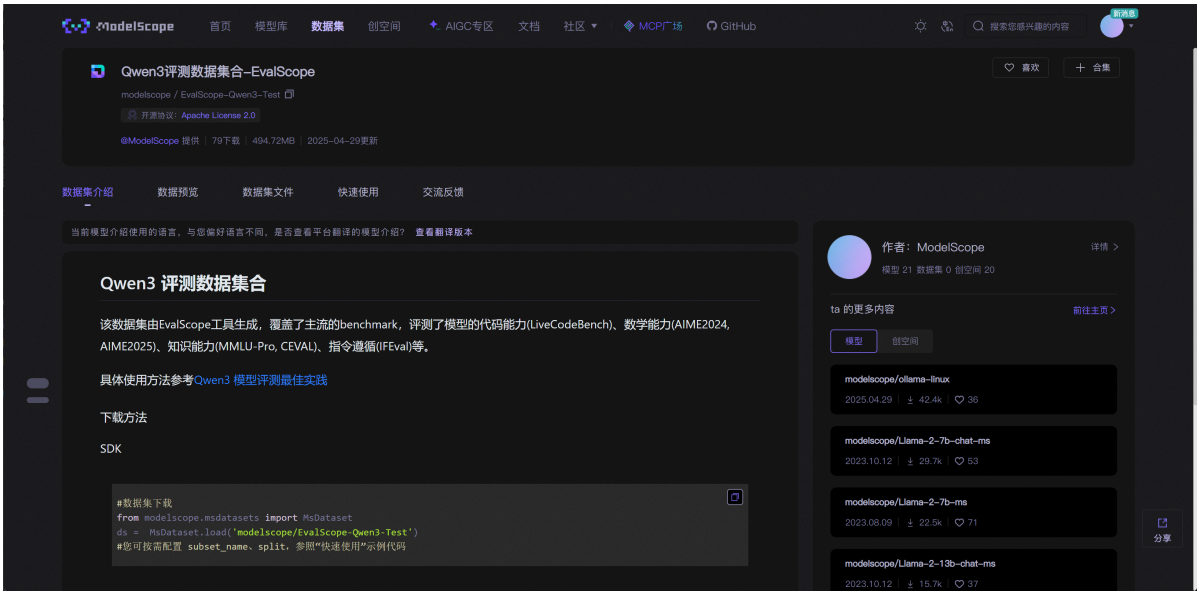


评测代码：

```
from evalscope import TaskConfig, run_task
task_cfg = TaskConfig(
    model='./Qwen3-32B-unsloth-bnb-4bit',
    api_url='http://127.0.0.1:8000/v1/chat/completions',
    eval_type='service',
    datasets=[
        'data_collection',
    ],
    dataset_args={
        'data_collection': {
            'dataset_id': 'modelscope/EvalScope-Qwen3-Test',
            'filters': {'remove_until': '</think>'}  # 过滤掉思考的内容
        }
    },
    eval_batch_size=128,
    generation_config={
        'max_tokens': 30000,  # 最大生成token数，建议设置为较大值避免输出截断
        'temperature': 0.6,  # 采样温度 (qwen 报告推荐值)
        'top_p': 0.95,  # top-p采样 (qwen 报告推荐值)
        'top_k': 20,  # top-k采样 (qwen 报告推荐值)
        'n': 1,  # 每个请求产生的回复数量
    },
    timeout=60000,  # 超时时间
    stream=True,  # 是否使用流式输出
    limit=2000,  # 设置为2000条数据进行测试
)

run_task(task_cfg=task_cfg)
```

这里采用了EvalScope专门为Qwen3准备的`modelscope/EvalScope-Qwen3-Test`数据集进行评测，会围绕模型的推理、指令跟随、代理能力和多语言支持方面能力进行测试，该数据包含`mmlu_pro`、`ifeval`、`live_code_bench`、`math_500`、`aime24`等各著名评估数据集。

数据集地址：https://modelscope.cn/datasets/modelscope/EvalScope-Qwen3-Test/summary



评测运行结果如下：

```python
from evalscope import TaskConfig, run_task
task_cfg = TaskConfig(
    model='./Qwen3-32B-unsloth-bnb-4bit',
    api_url='http://127.0.0.1:8000/v1/chat/completions',
    eval_type='service',
    datasets=[
        'data_collection',
    ],
    dataset_args={
        'data_collection': {
            'dataset_id': 'modelscope/EvalScope-Qwen3-Test',
            'filters': {'remove_until': '</think>'}  # 过滤掉思考的内容
        }
    },
    eval_batch_size=128,
    generation_config={
        'max_tokens': 30000,  # 最大生成token数，建议设置为较大值避免输出截断
        'temperature': 0.6,  # 采样温度 (qwen 报告推荐值)
        'top_p': 0.95,  # top-p采样 (qwen 报告推荐值)
        'top_k': 20,  # top-k采样 (qwen 报告推荐值)
        'n': 1,  # 每个请求产生的回复数量
    },
    timeout=60000,  # 超时时间
    stream=True,  # 是否使用流式输出
    limit=100,  # 设置为100条数据进行测试
)

run_task(task_cfg=task_cfg)
```

```
2025-05-05 01:54:44,011 - evalscope - INFO - Args: Task config is provided with TaskConfig type.
```

```
    "datasets": [
        "data_collection"
    ],
    "dataset_args": {
        "data_collection": {
            "dataset_id": "modelscope/EvalScope-Qwen3-Test",
            "filters": {
                "remove_until": "</think>"
            }
        }
    },
    "dataset_dir": "/root/.cache/modelscope/hub/datasets",
    "dataset_hub": "modelscope",
    "generation_config": {
        "max_tokens": 30000,
        "temperature": 0.6,
        "top_p": 0.95,
        "top_k": 20,
        "n": 1
    },
    "eval_type": "service",
    "eval_backend": "Native",
    "eval_config": null,
    "stage": "all",
    "limit": 100,
    "eval_batch_size": 128,
    "mem_cache": false,
    "use_cache": null,
    "work_dir": "./outputs/20250505_015444",
    "outputs": null,
    "debug": false,
    "dry_run": false,
    "seed": 42,
    "api_url": "http://127.0.0.1:8000/v1/chat/completions",
    "api_key": "EMPTY",
```


此时后台显示如下：


```
, stop=[], stop_token_ids=[], bad_words=[], include_stop_str_in_output=False, ignore_eos=False, max_tokens=30000, min_tokens=0, log
probs=None, prompt_logprobs=None, skip_special_tokens=True, spaces_between_special_tokens=True, truncate_prompt_tokens=None, guided
_decoding=None, extra_args=None), prompt_token_ids: None, lora_request: None, prompt_adapter_request: None.
INFO:     127.0.0.1:34624 - "POST /v1/chat/completions HTTP/1.1" 200 OK
INFO 05-05 17:15:35 [async_llm.py:252] Added request chatcmpl-316e29c20bb348889dd3af633b4aaddb.
INFO 05-05 17:15:36 [logger.py:39] Received request chatcmpl-84f2ebac07764976b6a5bfd170ef5916: prompt: '<|im_start|>user\nThe follo
wing are multiple choice questions (with answers) about engineering. Think step by step and then finish your answer with "the answe
r is (X)" where X is the correct letter choice.\nQ: A vessel with a frictionless piston contains 1 ft^3 of a gas at 200psia.The pis
ton is held in position by applying an externalforce. Calculate the amount of work done if: a) This external force is released grad
ually until the volume isdoubled in such a way that PV is always a constant. b) If the external force is released suddenly to half
its initial value.\nOptions are:\n(A): 19,930(ft-lb_f), 14,400(ft-lb_f)\n(B): 10,000(ft-lb_f), 7,000(ft-lb_f)\n(C): 17,500(ft-lb_f)
, 12,250(ft-lb_f)\n(D): 18,000(ft-lb_f), 13,500(ft-lb_f)\n(E): 22,000(ft-lb_f), 16,500(ft-lb_f)\n(F): 20,500(ft-lb_f), 15,375(ft-lb
_f)\n(G): 15,000(ft-lb_f), 10,000(ft-lb_f)\n(H): 20,000(ft-lb_f), 15,000(ft-lb_f)\n(I): 19,500(ft-lb_f), 14,625(ft-lb_f)\n(J): 21,0
00(ft-lb_f), 15,750(ft-lb_f)\n\n<|im_end|>\n<|im_start|>assistant\n', params: SamplingParams(n=1, presence_penalty=0.0, frequency_p
enalty=0.0, repetition_penalty=1.0, temperature=0.6, top_p=0.95, top_k=20, min_p=0.0, seed=None, stop=[], stop_token_ids=[], bad_wo
rds=[], include_stop_str_in_output=False, ignore_eos=False, max_tokens=30000, min_tokens=0, logprobs=None, prompt_logprobs=None, sk
ip_special_tokens=True, spaces_between_special_tokens=True, truncate_prompt_tokens=None, guided_decoding=None, extra_args=None), pr
ompt_token_ids: None, lora_request: None, prompt_adapter_request: None.
INFO:     127.0.0.1:34636 - "POST /v1/chat/completions HTTP/1.1" 200 OK
INFO 05-05 17:15:36 [async_llm.py:252] Added request chatcmpl-84f2ebac07764976b6a5bfd170ef5916.
INFO 05-05 17:15:37 [loggers.py:111] Engine 000: Avg prompt throughput: 0.0 tokens/s, Avg generation throughput: 780.9 tokens/s, Ru
nning: 106 reqs, Waiting: 22 reqs, GPU KV cache usage: 100.0%, Prefix cache hit rate: 21.1%
```

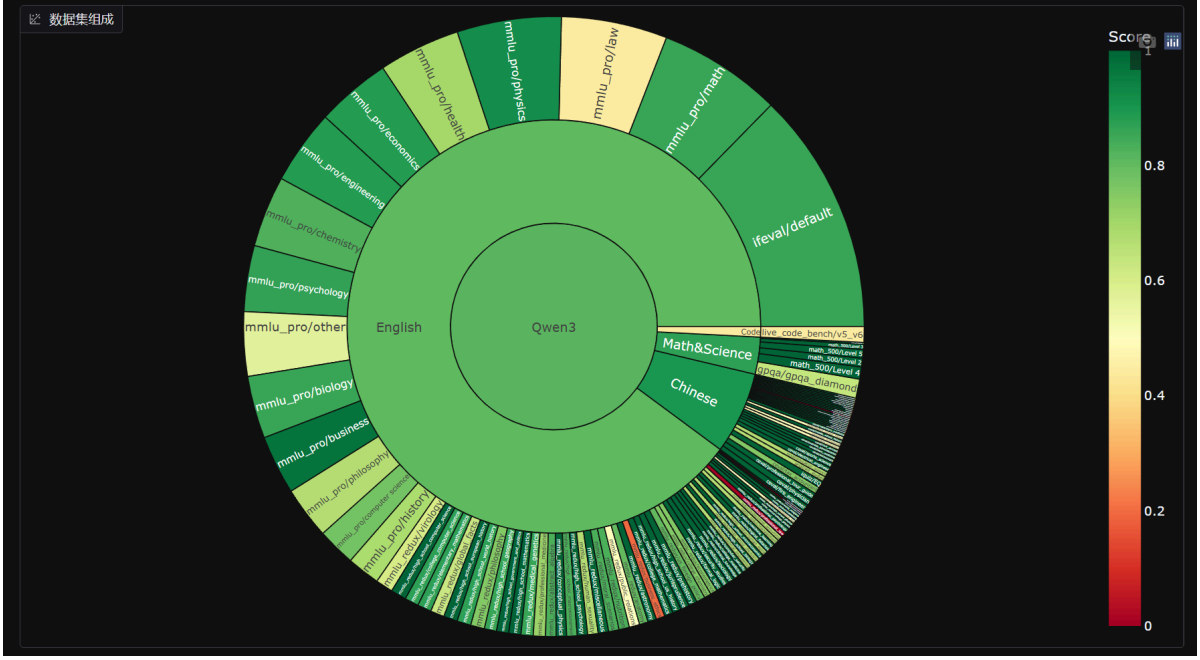

查看模型能力评测结果

```
evalscope app
```

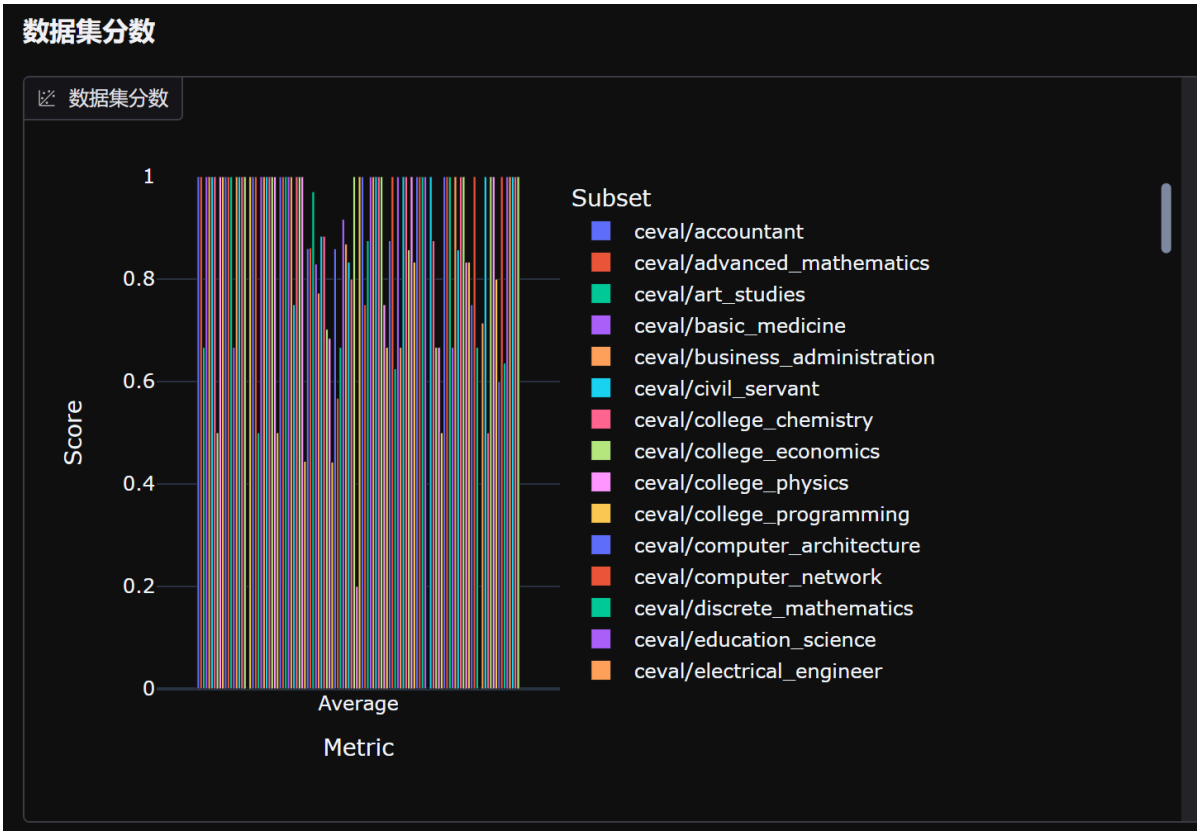
```
(evalscope) root@autodl-container-b4de43b42c-6d000661:~/autodl-tmp/Qwen3# evalscope app
* Running on local URL:  http://0.0.0.0:7860

To create a public link, set `share=True` in `launch()`.
```

对评测结果进行可视化：

数据集组成


数据集组成
Qwen3
English
Math&Science
Chinese
ifeval/default
mmlu_pro/math
mmlu_pro/law
mmlu_pro/physics
mmlu_pro/health
mmlu_pro/economics
mmlu_pro/engineering
mmlu_pro/chemistry
mmlu_pro/psychology
mmlu_pro/other
mmlu_pro/biology
mmlu_pro/business
mmlu_pro/philosophy
mmlu_pro/history
gpqa/gpqa_diamond
Score
1
0.8
0.6
0.4
0.2
0

数据集分数


数据集分数
Score
1
0.8
0.6
0.4
0.2
0
Average
Metric
Subset
ceval/accountant
ceval/advanced_mathematics
ceval/art_studies
ceval/basic_medicine
ceval/business_administration
ceval/civil_servant
ceval/college_chemistry
ceval/college_economics
ceval/college_physics
ceval/college_programming
ceval/computer_architecture
ceval/computer_network
ceval/discrete_mathematics
ceval/education_science
ceval/electrical_engineer

数据集分数表

Model	Dataset	Metric	Subset	Num	Score	C
Qwen3-32B-unsloth-bnb-4bit	EvalScope-Qwen3-Test	Average	ceval/accountant	2	1.0000	Q
Qwen3-32B-unsloth-bnb-4bit	EvalScope-Qwen3-Test	Average	ceval/advanced_mathematics	1	1.0000	Q
Qwen3-32B-unsloth-bnb-4bit	EvalScope-Qwen3-Test	Average	ceval/art_studies	3	0.6667	Q
Qwen3-32B-unsloth-bnb-4bit	EvalScope-Qwen3-Test	Average	ceval/basic_medicine	2	1.0000	Q
Qwen3-32B-unsloth-bnb-4bit	EvalScope-Qwen3-Test	Average	ceval/business_administration	2	1.0000	Q
Qwen3-32B-unsloth-bnb-4bit	EvalScope-Qwen3-Test	Average	ceval/civil_servant	2	1.0000	Q
Qwen3-32B-unsloth-bnb-4bit	EvalScope-Qwen3-Test	Average	ceval/college_chemistry	1	1.0000	Q
Qwen3-32B-unsloth-bnb-4bit	EvalScope-Qwen3-Test	Average	ceval/college_economics	2	0.5000	Q
Qwen3-32B-unsloth-bnb-4bit	EvalScope-Qwen3-Test	Average	ceval/college_physics	1	1.0000	Q
Qwen3-32B-unsloth-bnb-4bit	EvalScope-Qwen3-Test	Average	ceval/college_programming	1	1.0000	Q
Qwen3-32B-unsloth-bnb-4bit	EvalScope-Qwen3-Test	Average	ceval/computer_architecture	1	1.0000	Q
Qwen3-32B-unsloth-bnb-4bit	EvalScope-Qwen3-Test	Average	ceval/computer_network	1	1.0000	Q

4. wandb安装与注册

4.1 wandb基本说明

在大规模模型训练中，我们往往需要监控和分析大量的训练数据，而WandB可以帮助我们实现这一目标。它提供了以下几个重要的功能：

实时可视化：WandB可以实时展示训练过程中关键指标的变化，如损失函数、学习率、训练时间等。通过这些可视化数据，我们能够直观地了解模型的训练进展，快速发现训练中的异常或瓶颈。

自动记录与日志管理：WandB会自动记录每次实验的参数、代码、输出结果，确保实验结果的可追溯性。无论是超参数的设置，还是模型的架构调整，WandB都能够帮助我们完整保留实验记录，方便后期对比与调优。

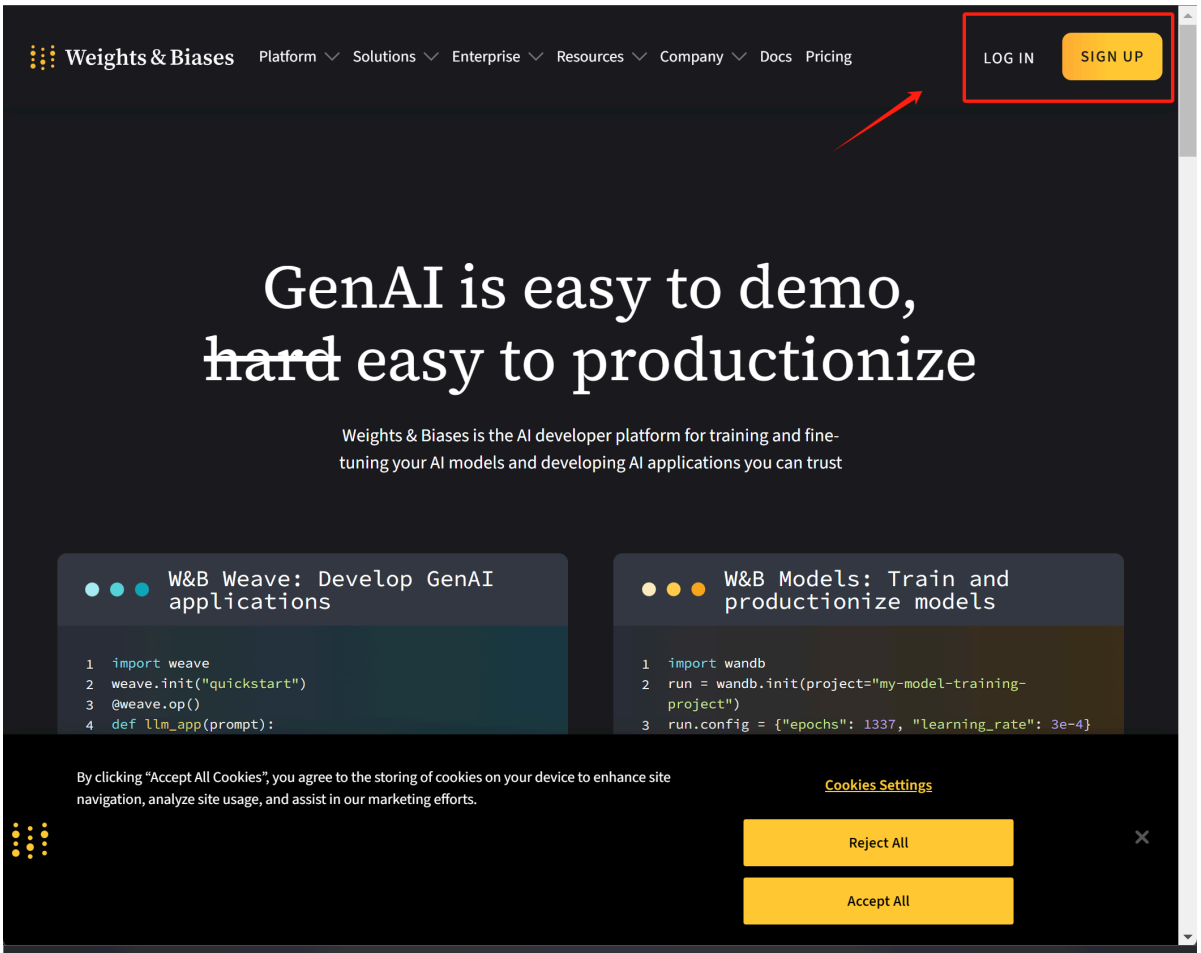
支持中断与恢复训练：在长时间的预训练任务中，系统中断或需要暂停是常见的情况。通过WandB的checkpoint功能，我们可以随时恢复训练，从上次中断的地方继续进行，避免数据和时间的浪费。

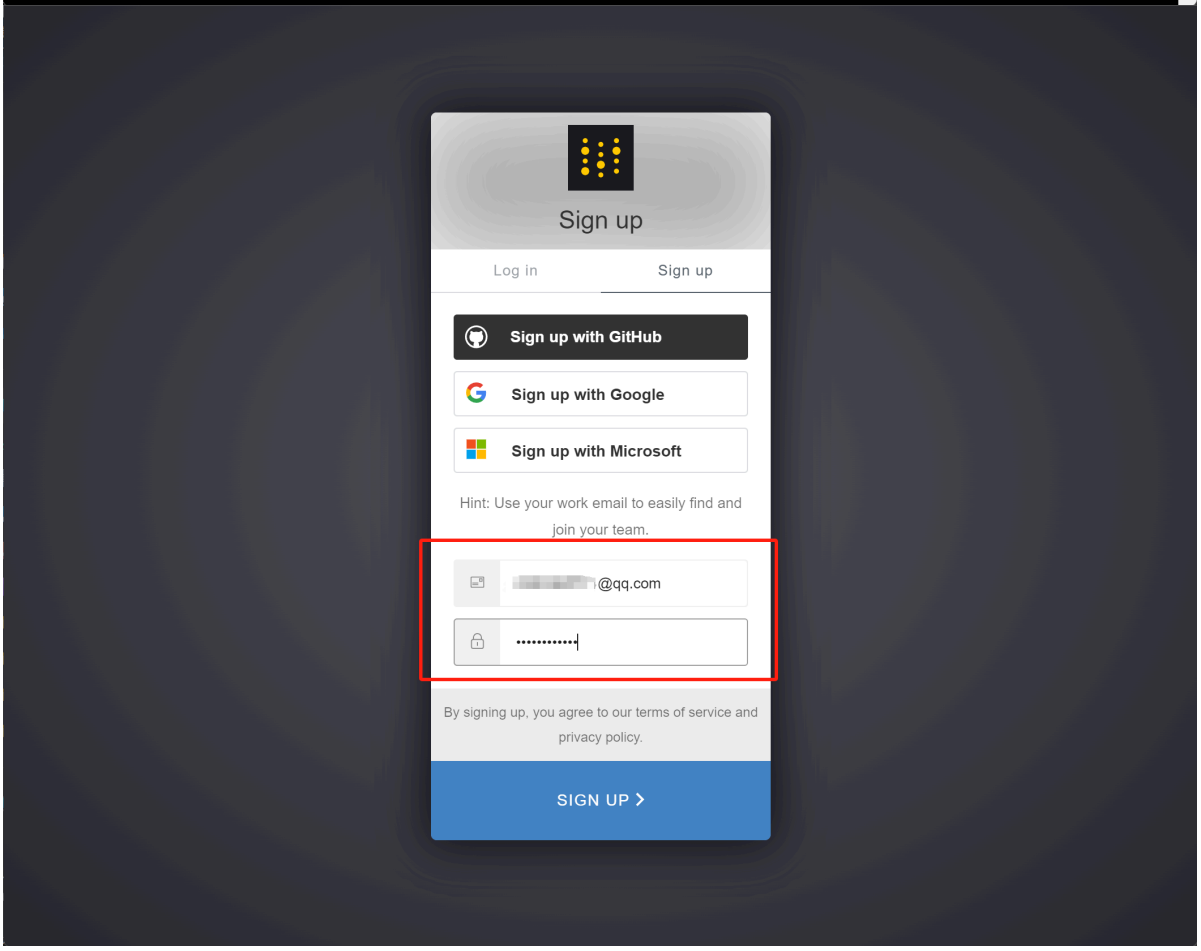
多实验对比：当我们尝试不同的模型配置或超参数时，WandB允许我们在多个实验之间轻松进行对比分析，帮助我们选择最优的模型配置。

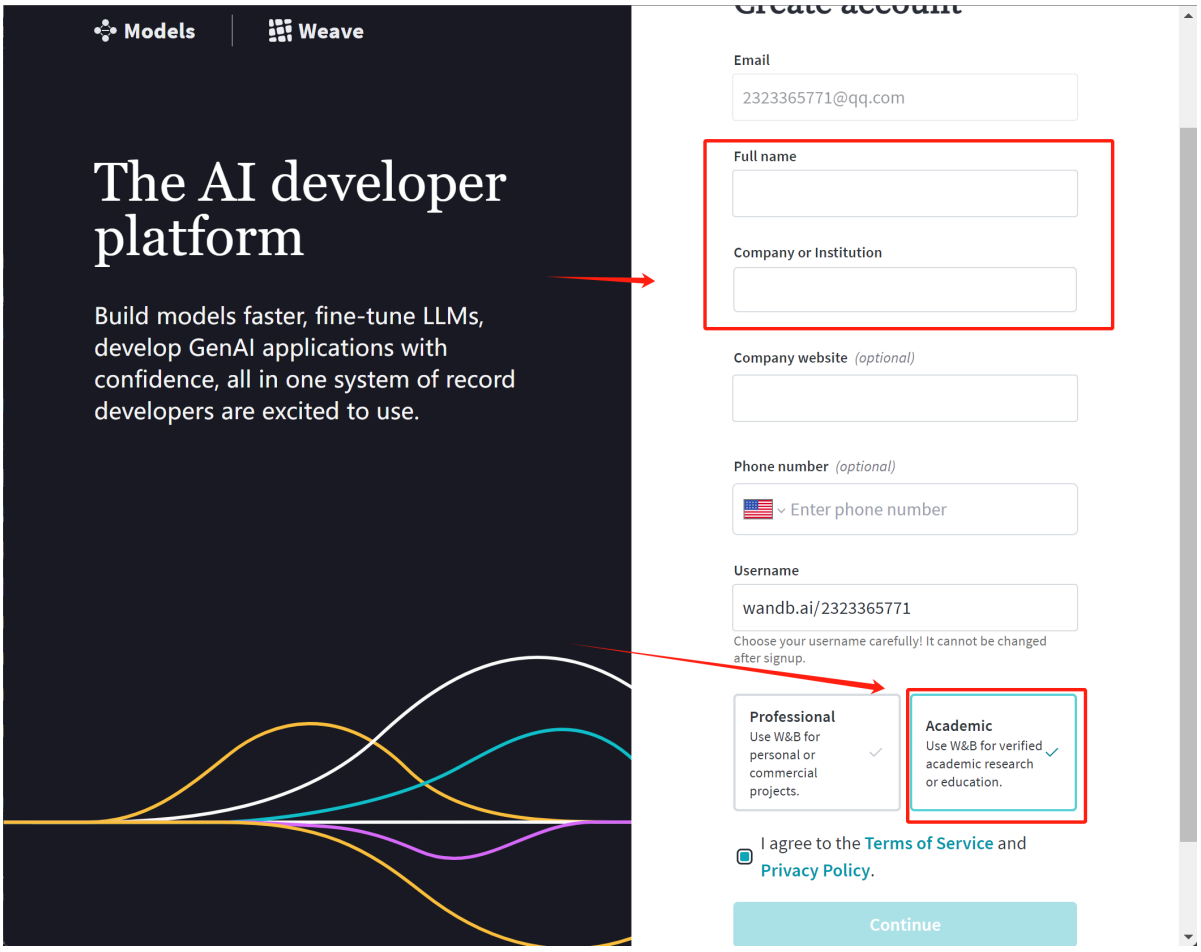
团队协作：WandB还支持团队协作，多个成员可以共同查看实验结果，协同调试模型。这对研究和项目开发中团队的合作非常有帮助。

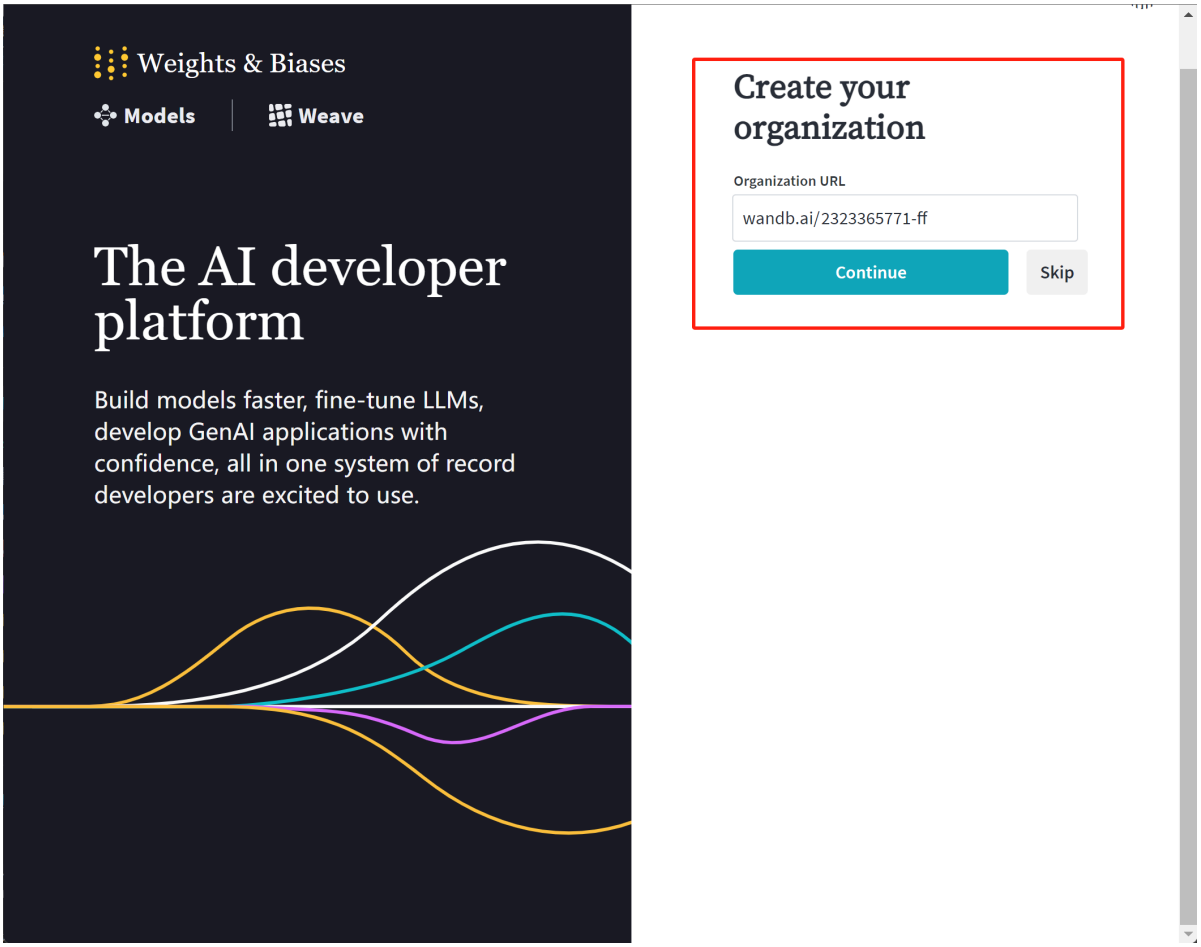
4.2 wandb注册与使用

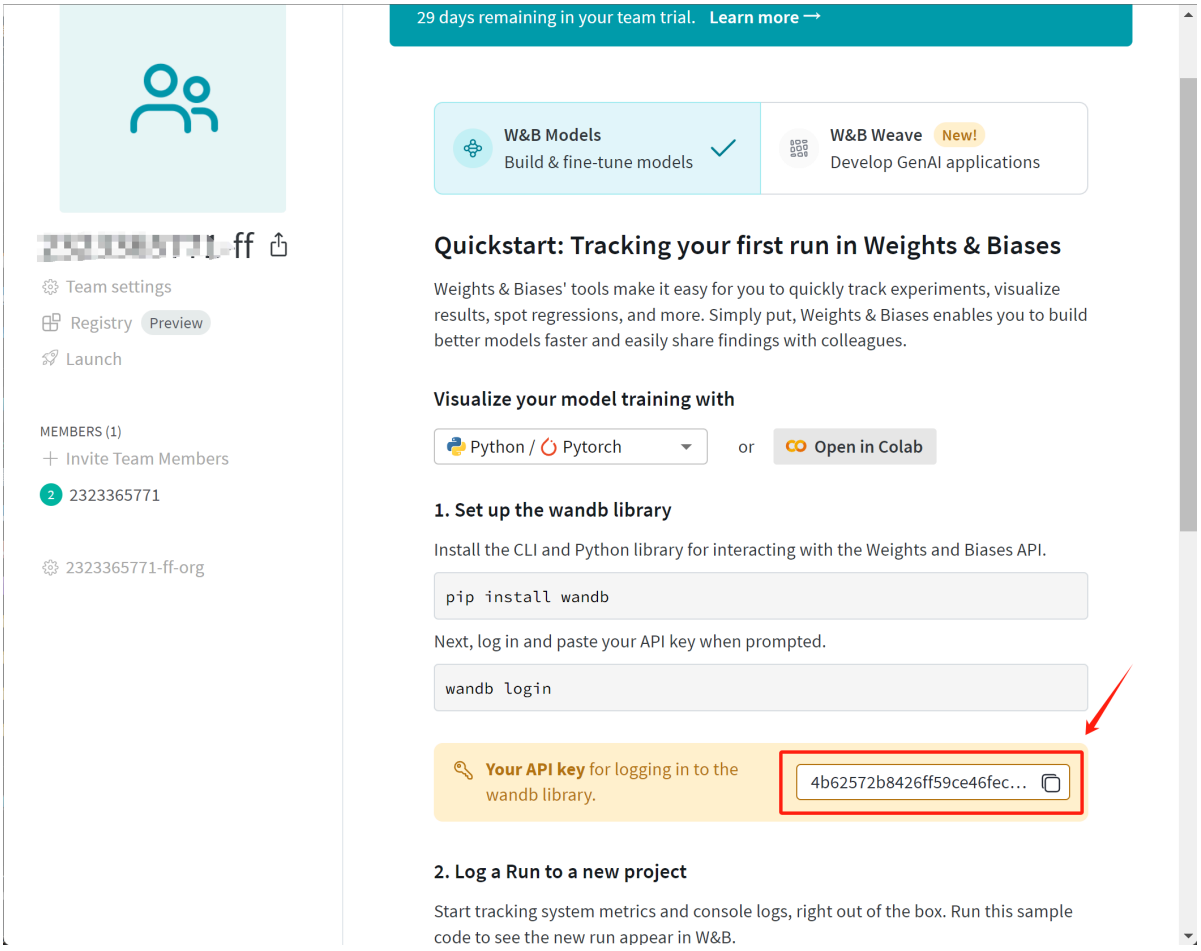
wandb官网：https://wandb.ai/site


Weights & Biases
Platform
Solutions
Enterprise
Resources
Company
Docs
Pricing
LOG IN
SIGN UP
GenAI is easy to demo,
~~hard~~ easy to productionize
Weights & Biases is the AI developer platform for training and fine-tuning your AI models and developing AI applications you can trust
W&B Weave: Develop GenAI applications
1 import weave
2 weave.init("quickstart")
3 @weave.op()
4 def llm_app(prompt):
W&B Models: Train and productionize models
1 import wandb
2 run = wandb.init(project="my-model-training-project")
3 run.config = {"epochs": 1337, "learning_rate": 3e-4}
By clicking "Accept All Cookies", you agree to the storing of cookies on your device to enhance site navigation, analyze site usage, and assist in our marketing efforts.
Cookies Settings
Reject All
Accept All


Sign up
Log in
Sign up
Sign up with GitHub
Sign up with Google
Sign up with Microsoft
Hint: Use your work email to easily find and join your team.
@qq.com
By signing up, you agree to our terms of service and privacy policy.
SIGN UP >


Models
Weave
The AI developer platform
Build models faster, fine-tune LLMs, develop GenAI applications with confidence, all in one system of record developers are excited to use.
Email
2323365771@qq.com
Full name
Company or Institution
Company website (optional)
Phone number (optional)
Enter phone number
Username
wandb.ai/2323365771
Choose your username carefully! It cannot be changed after signup.
Professional
Use W&B for personal or commercial projects.
Academic
Use W&B for verified academic research or education.
I agree to the Terms of Service and Privacy Policy.
Continue


Weights & Biases
Models
Weave
The AI developer platform
Build models faster, fine-tune LLMs, develop GenAI applications with confidence, all in one system of record developers are excited to use.
Create your organization
Organization URL
wandb.ai/2323365771-ff
Continue
Skip

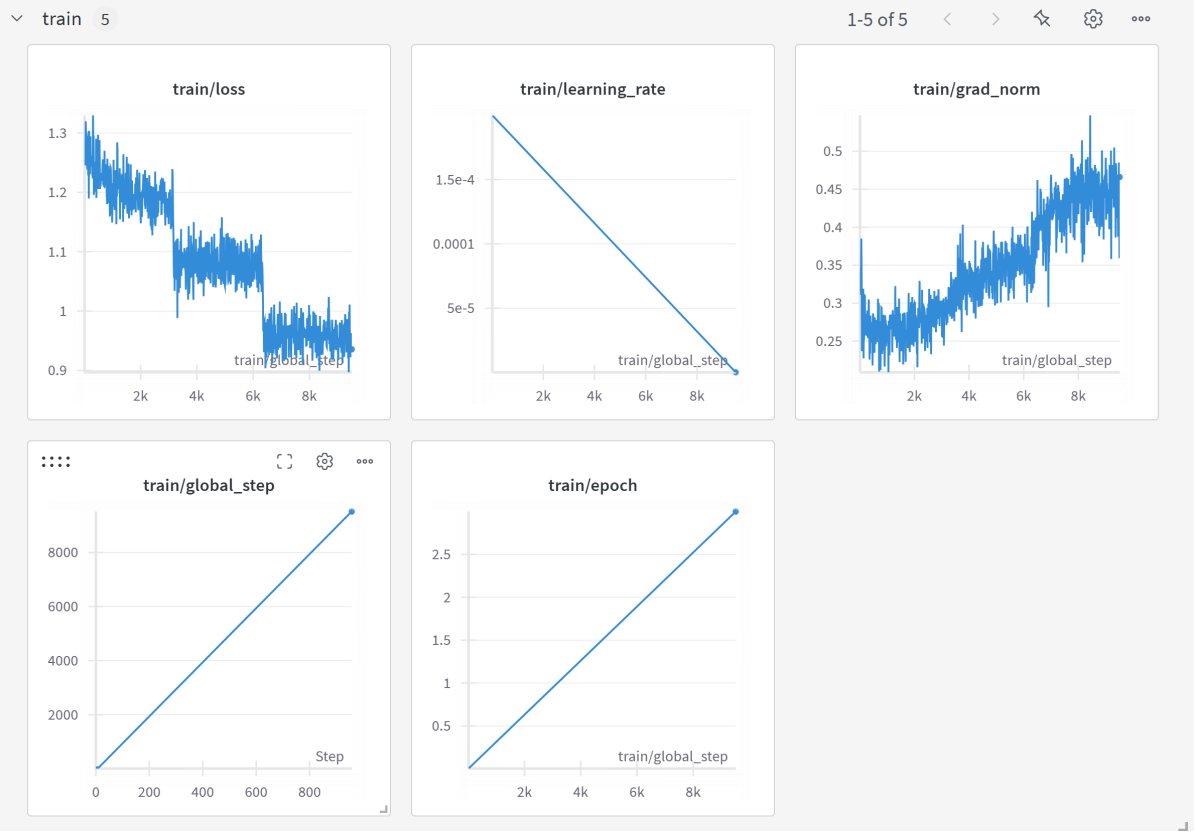


然后即可在令行中输入如下代码安装wandb：

```
pip install wandb
```

```
(MateConv) root@autodl-container-3b9d4a8a31-65ad2c6b:~/autodl-tmp/MateConv# cd /root/autodl-tmp/MateConv/model
(MateConv) root@autodl-container-3b9d4a8a31-65ad2c6b:~/autodl-tmp/MateConv/model# pip install wandb
Looking in indexes: http://mirrors.aliyun.com/pypi/simple
Collecting wandb
  Downloading http://mirrors.aliyun.com/pypi/packages/e2/ae/943335dd9b8783db59133047a1fdf53f5070cadab7ce2464632e36ba6b7f
/wandb-0.18.5-py3-none-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (16.0 MB)
     ---------------------------------------- 16.0/16.0 MB 6.3 MB/s eta 0:00:00
Requirement already satisfied: click!=8.0.0,>=7.1 in /root/miniconda3/envs/MateConv/lib/python3.10/site-packages (from w
andb) (8.1.7)
Collecting docker-pycreds>=0.4.0 (from wandb)
  Downloading http://mirrors.aliyun.com/pypi/packages/f5/e8/f6bd1eee09314e7e6dee49cbe2c5e22314ccdb38db16c9fc72d2fa80d054
/docker_pycreds-0.4.0-py2.py3-none-any.whl (9.0 kB)
Requirement already satisfied: gitpython!=3.1.29,>=1.0.0 in /root/miniconda3/envs/MateConv/lib/python3.10/site-packages
(from wandb) (3.1.43)
Requirement already satisfied: platformdirs in /root/miniconda3/envs/MateConv/lib/python3.10/site-packages (from wandb)
```

接下来在unsloth微调前，我们即可设置wandb进行微调记录，并可在对应网站上观察到训练过程如下：

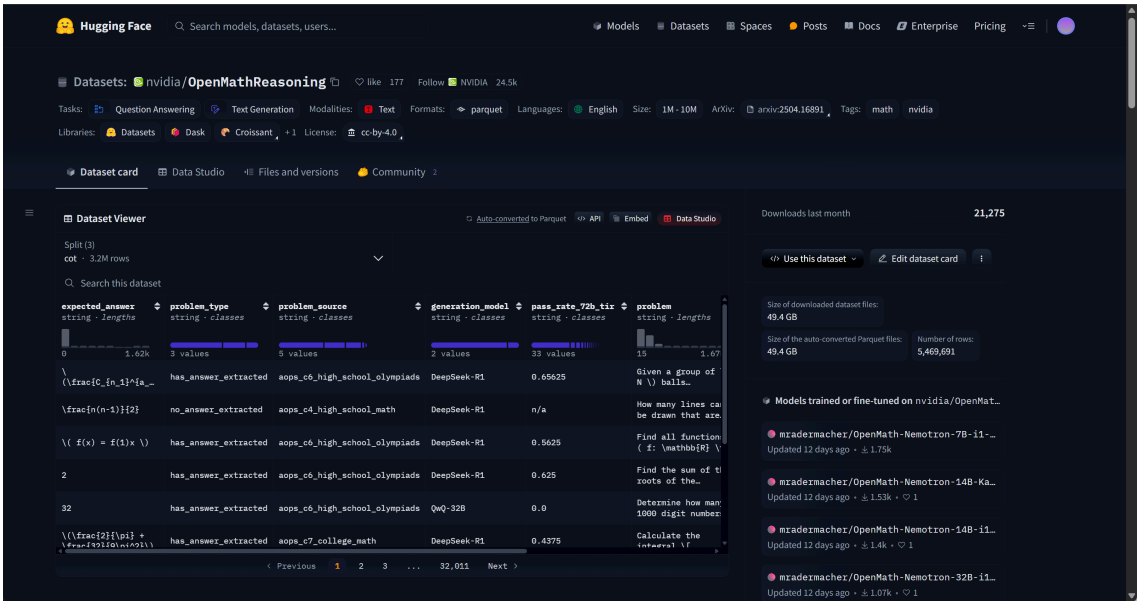


三、Qwen3混合推理模型微调数据集准备

在搭建好了基础环境后，接下来即可开始着手准备Qwen3微调数据集了。围绕Qwen3模型的高效微调，为了确保其仍然保留混合推理能力，我们可以考虑在微调数据集中加入如普通对话数据集FineTome（https://huggingface.co/datasets/mlabonne/FineTome-100k），以及带有推理字段的数学类数据集OpenMathReasoning（https://huggingface.co/datasets/unsloth/OpenMathReasoning-mini），并围绕这两个数据集进行拼接，从而在确保能提升模型的数学能力的同时，保留非推理的功能。同时还需要在持续微调训练过程中不断调整COT数学数据集和普通文本问答数据集之间的配比，以确保模型能够在提升数学能力的同时，保留混合推理的性能。

1. OpenMathReasoning 数据集（AIMO-2）

- 数据集地址：https://huggingface.co/datasets/unsloth/OpenMathReasoning-mini



数据集介绍：OpenMathReasoning 是为 AI Mathematical Olympiad - Progress Prize 2（AIMO-2）竞赛开发的高质量数学推理数据集。该数据集包含：

- **540,000** 道独特的高质量数学问题，涵盖代数、组合、几何和数论等领域；
- **3,200,000** 条详细的长推理解答；
- **1,700,000** 条工具集成推理（Tool-Integrated Reasoning）解答，结合了代码执行与推理过程；
- **生成式解答选择（GenSelect）** 方法，用于从多个候选解答中选择最优解。

该数据集旨在推动 AI 在复杂数学推理任务中的能力提升，已被用于训练在多个数学推理基准上表现优异的模型。

- 数据集格式

OpenMathReasoning包含**expected_answer**、**problem**和**generated_solution**三个核心字段，分别代表问题答案、问题和模型思考过程。例如其中一条数据内容如下：

问题：

> 设有一组共 NN 个球，由 CC 种颜色组成，其中每种颜色的球的数量分别为 n1,n2,…,nCn_1, n_2, \ldots, n_C（满足 n1+n2+…+nC=Nn_1 + n_2 + \ldots + n_C = N）。如果从中随机抽取 AA 个球（其中 A≤NA \leq N），那么抽到的球中恰好有 a1,a2,…,aCa_1, a_2, \ldots, a_C 个分别对应每种颜色（满足 a1+a2+…+aC=Aa_1 + a_2 + \ldots + a_C = A），这种情况发生的概率是多少？

答案：

$$\frac{C_{n_1}^{a_1} \cdot C_{n_2}^{a_2} \cdot \ldots \cdot C_{n_C}^{a_C}}{C_N^A}$$

思考过程：

> 好的，也就是说我要计算这样一个概率：从总共 N 个球中随机抽取 A 个球，其中球分为 C 种颜色，要求抽到的每种颜色球的数量恰好是 $a_1, a_2, \ldots, a_C$。嗯，我们来想想该怎么求解这个问题。
>
> 首先，概率问题通常会涉及**组合数**。概率的一般公式是：
> **有利情况数 ÷ 所有可能情况总数**。
>
> 在本题中，“有利情况”就是指：恰好抽到 a_1 个颜色1的球，a_2 个颜色2的球，依此类推，一直到颜色 C。
> 而“所有可能情况”则是：从 N 个球中任意抽取 A 个球的方式，不考虑颜色分布。
>
> 我们可以这样分解这个问题：
>
> * 从 N 个球中任意抽取 A 个球的总方式数是组合数：
>
> $$
>   C(N, A) = \binom{N}{A} = \frac{N!}{A!(N - A)!}
> $$
>
> * 而有利情况则是：从每种颜色中选出指定数量的球。比如从 n_1 个颜色1的球中选出 a_1 个，从 n_2 个颜色2的球中选出 a_2 个，依此类推。
>   由于每种颜色之间的选择是相互独立的，所以所有组合方式的总数就是各个颜色组合数的乘积：
>
> $$
>   \binom{n_1}{a_1} \times \binom{n_2}{a_2} \times \cdots \times \binom{n_C}{a_C}
> $$
>
> 不过，要注意前提条件必须满足：

* 每个 $a_i \leq n_i$，也就是说不能从某种颜色中选出比它实际数量更多的球；
* 同时 $a_1 + a_2 + \cdots + a_C = A$，总共抽取的球数必须正确。

如果这些条件满足，那么这种组合方式就是有效的。如果不满足，比如某个 $a_i > n_i$，则该组合数为0，概率也自然为0，这是符合逻辑的。

其实这正是一个**多项超几何分布（multivariate hypergeometric distribution）**问题。

超几何分布描述的是：从有限总体中不放回地抽取样本，得到某个类别成功次数的概率。多项超几何分布是其推广，适用于多个类别的情况，正好契合这里的“多种颜色”的设定。

该分布的概率公式就是：

$$
P = \frac{\binom{n_1}{a_1} \cdot \binom{n_2}{a_2} \cdots \binom{n_C}{a_C}}{\binom{N}{A}}
$$

最终结论：

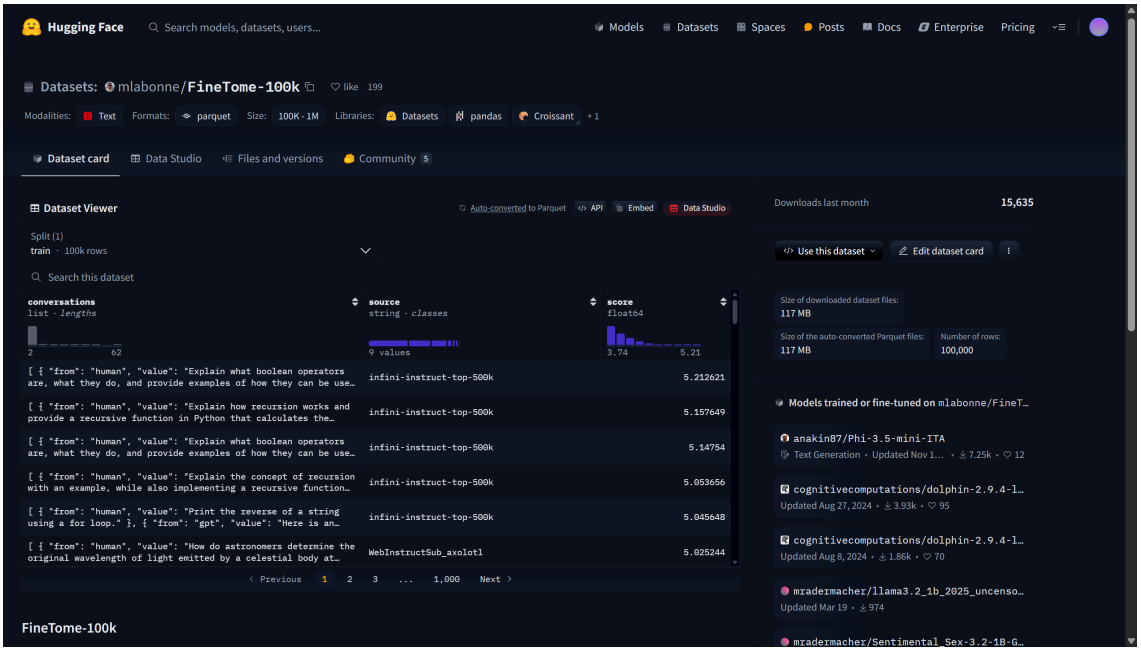
当从 N 个球（含有 C 种颜色，每种颜色分别有 $n_1, n_2, \ldots, n_C$ 个球）中随机抽取 A 个球时，恰好抽中每种颜色 $a_1, a_2, \ldots, a_C$ 个球的概率为：

$$
\boxed{
P = \frac{\prod_{i=1}^{C} \binom{n_i}{a_i}}{\binom{N}{A}}
}
$$

这个公式就是多项超几何分布的概率表达式，分子为每种颜色的有利组合方式之乘积，分母为所有可能抽取 A 个球的方式总数。

2. FineTome-100k 数据集（Maxime Labonne）

- 数据集地址：https://huggingface.co/datasets/mlabonne/FineTome-100k



数据集简介：FineTome-100k 是由 Maxime Labonne 创建的高质量多轮对话数据集，采用 ShareGPT 风格，适用于大语言模型的微调。该数据集特点包括：

- **100,000** 条多轮对话样本；
- 数据以 JSONL 格式存储，每条记录包含一个 "conversations" 字段，记录对话的完整历史；
- 对话格式类似于 ShareGPT，适合训练模型进行多轮对话；
- 可转换为 Hugging Face 通用的多轮对话格式，以适配不同的训练框架。

文件名	大小	类型	修改时间	权限	用户/用户组
.ipynb_checkpoints		文件夹	2025/05/05 17:37	drwxr-xr-x	root/root
cleaned_qwen3_dataset		文件夹	2025/05/05 19:02	drwxr-xr-x	root/root
dataset		文件夹	2025/05/04 17:19	drwxr-xr-x	root/root
evalscope		文件夹	2025/05/05 16:44	drwxr-xr-x	root/root
ktransformers		文件夹	2025/04/29 18:24	drwxr-xr-x	root/root
outputs		文件夹	2025/05/06 12:29	drwxr-xr-x	root/root
Qwen3-1.7B		文件夹	2025/05/05 00:11	drwxr-xr-x	root/root
Qwen3-1.7B-unsloth-bnb-4bit		文件夹	2025/05/04 22:51	drwxr-xr-x	root/root

全部文件 / Qwen3微调实战　　已全部加载，共4个

文件名	大小	类型	修改时间
课件&代码	-	文件夹	2025.05.06 20:13
数据集	-	文件夹	2025.05.06 19:57
模型权重	-	文件夹	2025.05.06 18:39
微调后模型权重	-	文件夹	2025.05.06 14:49

接下来即可上手使用Unsloth进行高效微调了。

- 体验课内容节选自[《2025大模型Agent智能体开发实战》(5月班)]完整版付费课程

体验课时间有限，若想深度学习大模型技术，欢迎大家报名由我主讲的《2025大模型Agent智能体开发实战》(5月班)

《2025大模型Agent智能体开发实战》(5月班)

全网独家，大模型智能体开发硬核体系课
零基础直通大模型岗位要求

严格遵照大模型岗位能力要求设计
五大技术板块步步为营

Part 1.模型部署与调用

在线大模型 | 开源大模型 | 部署工具 | 加速工具

… …

Part 2.大模型微调技术

高效微调 | 强化学习微调 | 指令微调

… …

Part 3.大模型RAG技术

Native RAG | GraphRAG | RAG效果优化

… …

Part 4.大模型Agent开发技术

Coze、Dify | LangChain | LangGraph | AutoGen

… …

Part 5.大模型开发企业级案例实战

知识库问答 | 智能编程与AI数据分析

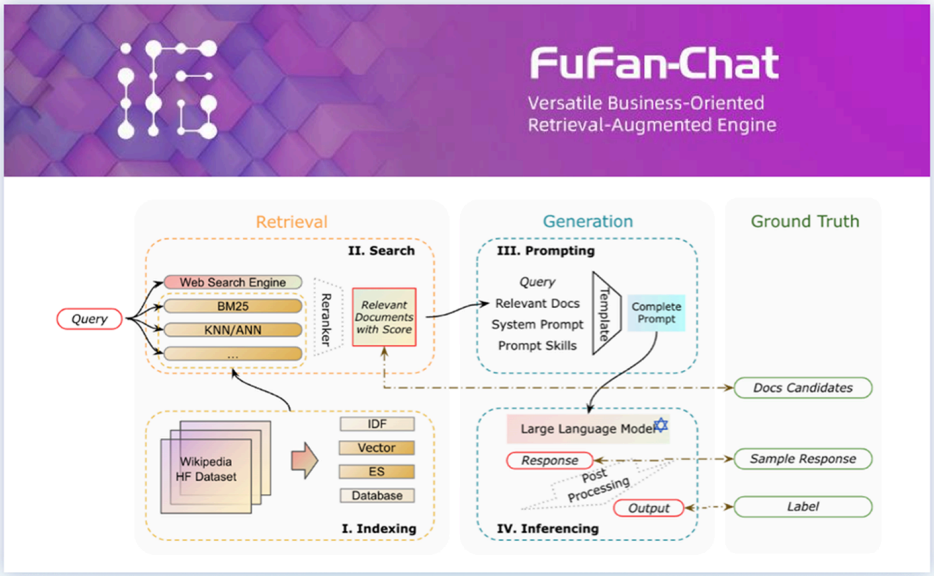
智能市场分析 | 智能客服

四大热门企业级智能体开发项目
百万研发成本独家实战案例

实战项目一

企业知识库问答项目实战

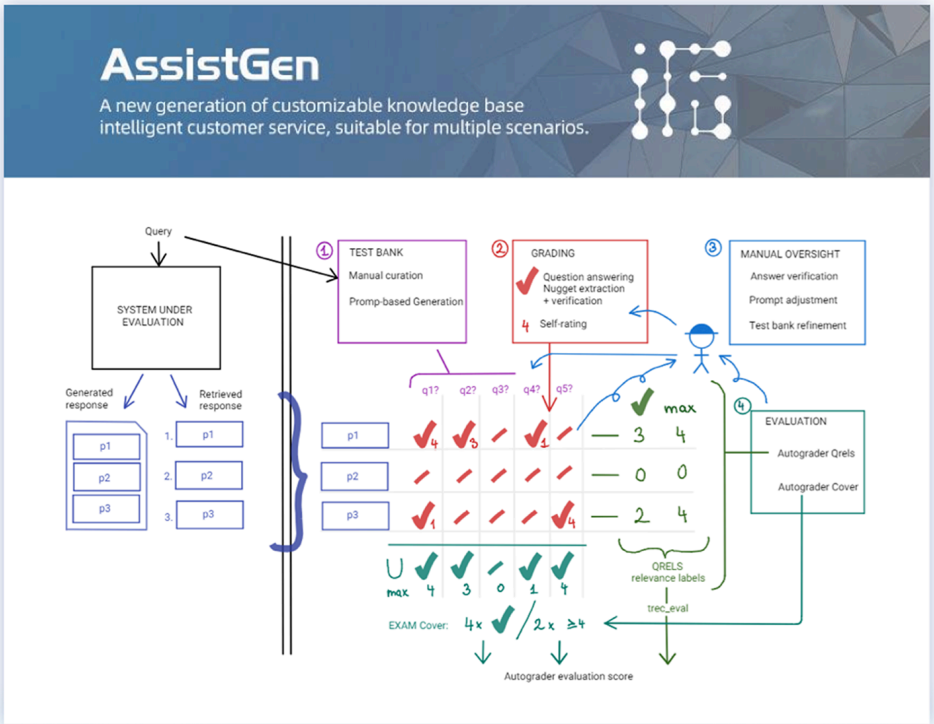
自研RAG框架 | 十万行代码顶级项目 | 超越开源RAG框架



实战项目二

智能客服Agent开发实战

自研企业级智能体 | 复杂知识库检索 | 自定义智能体编排

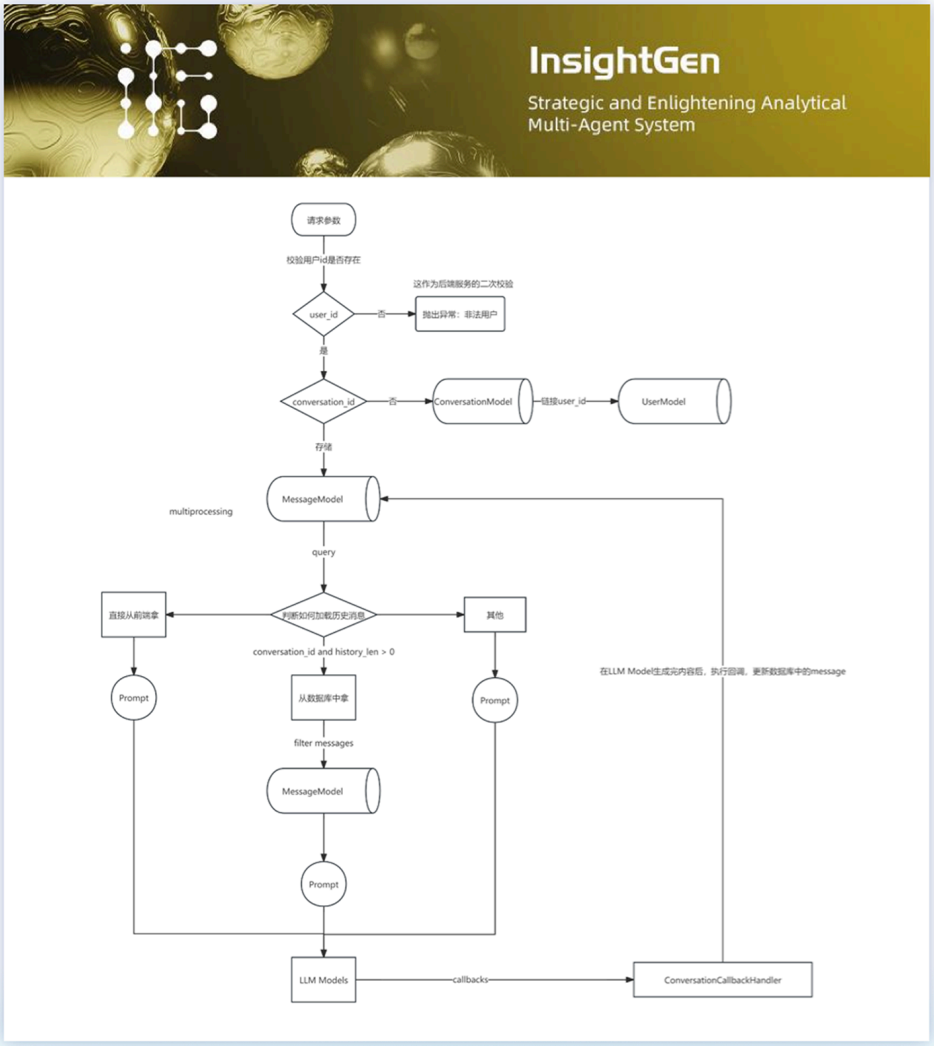


实战项目三

实战项目三

市场分析智能体开发实战

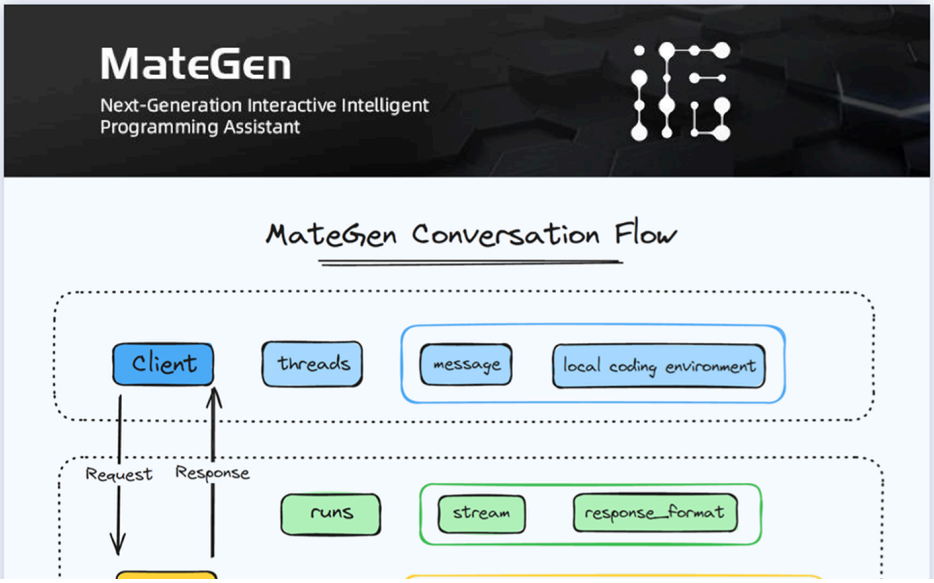
自研企业级智能体 | 复杂流程代理 | 多智能体协作

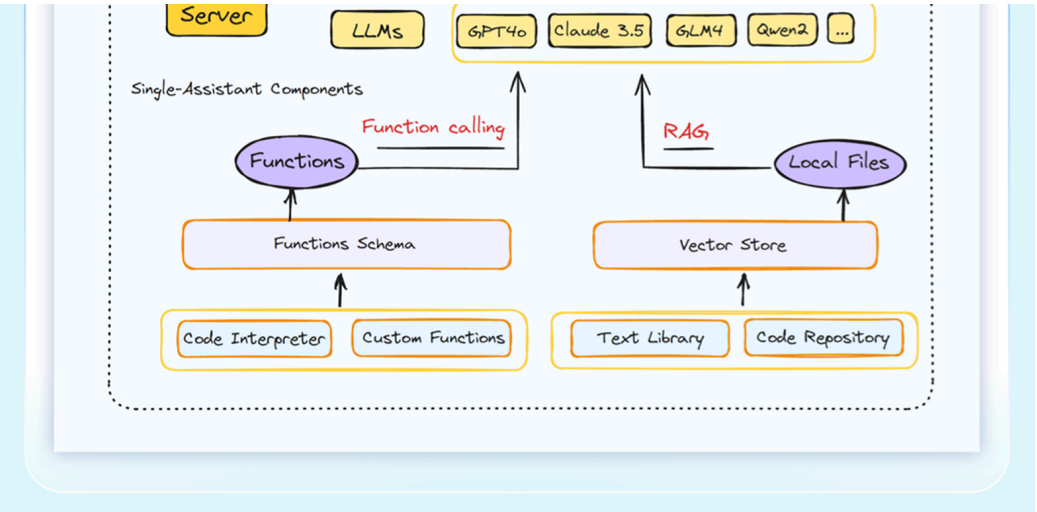


实战项目四

智能编程与数据数分智能MateGen项目开发实战

团队自研Agent | GitHub明星项目 | GPT4级实战性能





同时，5月班重磅新增DeepSeek+Agents SDK+谷歌ADK+MCP技术应用与智能体开发相关实战内容，并计划新增Qwen-3模型实战教学：

部分课程成果演示

- Dify+DeepSeek搭建智能客服

- 可视化数据分析Multi-Agent

- Ollama 自动化并发请求测试与动态资源监控

- Neo4j并行多线程导入百万级文本方法与实践

- MateGen Pro 项目功能演示

- 智能客服项目展示

此外，若是对大模型底层原理感兴趣，也欢迎报名由我和菜菜老师共同主讲的《2025大模型原理与实战课程》(5月班)

两门大模型课程5月班目前上新特惠中，立减2000起，合购还有更多优惠哦~详细信息扫码添加助教，回复“大模型”，即可领取课程大纲&查看课程详情👇